DESIGNING THE USER EXPERIENCE

Team Project – Deliverable 2

BUILDING THE APPLICATION

From wireframes to a working, global, accessible product

Implementation brief and submission template — team deliverable with individually graded contribution

Course	Designing the User Experience (ISTE.260)
Instructor	Aleksander Radovan
Academic year / semester	2026
Deliverable	Deliverable 2 – Implementation of the application (team)
Builds on	Deliverable 1 – Wireframes in Three Ways
Weight in the final grade	13,33%
Development period	From 19.10.20206. to 14.11.2026. , with a graded commit every week
Final submission deadline	14.11.2026. at 23:59h
Submission channel	MyCourses assignment

One document per team, submitted as PDF. Keep the .docx. Everything claimed in this document must be verifiable in the team's GitHub repository — the repository is the primary evidence, this document is the guided tour through it.

Contents

Contents 2
1. What this deliverable is 5
1.1 Learning outcomes 5
1.2 What is graded, and how often 5
2. The product you have to build 6
2.1 Scope 6
2.2 Minimum functional requirements 6
2.3 Definition of done 6
3. Repository and ways of working 7
3.1 Repository setup 7
3.2 Branching model 7
3.3 Weekly commit requirement 7
3.4 Commit messages and pull requests 7
3.5 Issues and planning 8
3.6 What we look at in the git history 8
4. Database requirement 9
5. External REST API requirement 9
5.1 Rules 9
5.2 Candidate APIs by topic area 9
6. Internationalization and localization 11
6.1 Required locales 11
6.2 Engineering rules (i18n) 11
6.3 Localization rules (L10n) 11
6.4 Layout rules 12
6.5 Cultural rules 12
7. Accessibility and universal design 13
7.1 Why this is not optional 13
7.2 Required practices 13
7.3 The ten-minute check 13
8. Weekly rhythm 15
9. Submission 15
9.1 What to submit 15
9.2 README requirements 15
9.3 Assessment 16
9.4 Academic integrity and AI use 16
1. Team, product and repository 17
1.1 Identification 17
1.2 Team members 17

1.3 The product in three sentences 18
1.4 Elevator demo path 18
2. Architecture and technology 20
2.1 Stack 20
2.2 Architecture diagram 20
2.3 Repository structure 21
3. From wireframes to screens 22
3.1 Merged screen map 22
3.2 What was dropped or merged, and why 23
3.3 Screens of the finished application 24
4. Database 28
4.1 Engine and justification 28
4.2 Entity-relationship diagram 28
4.3 Entities 29
4.4 Evidence that it works 30
5. External REST API integration 31
5.1 The service 31
5.2 Endpoints used 31
5.3 Failure handling 32
5.4 Caching and data volume 33
5.5 Evidence 33
6. Internationalization and localization 34
6.1 Locales implemented 34
6.2 How i18n is implemented 34
6.3 Plural rules 35
6.4 Locale-aware formatting 36
6.5 Same screen, three locales 38
6.6 Text expansion and layout 38
6.7 Direction and script 39
6.8 Cultural checks 40
7. Accessibility report 41
7.1 The ten-minute check 41
7.2 Tools used 42
7.3 Screen reader walkthrough 42
7.4 Contrast evidence 42
7.5 Known accessibility issues we did not fix 43
7.6 Inclusive design decisions 43
8. Git history and teamwork 44
8.1 Our branching model 44
8.2 Repository evidence 44

8.3 Commits per member per week 45
8.4 Pull requests 46
8.5 Honest account of the collaboration 46
9. Weekly progress log 48
10. Testing, quality and known issues 50
10.1 How the product was tested 50
10.2 Known bugs and limitations 50
10.3 What we would do next 51
11. Individual contribution statements 52
Member 1 52
Member 2 54
Member 3 55
Member 4 56
Member 5 57
12. AI usage declaration 58
13. Submission checklist 59

If the list above appears empty, click inside it and press F9 (Word) to update the field.

PART A — ASSIGNMENT BRIEF (read this first)

1. What this deliverable is

In Deliverable 1 each of you produced ten wireframes individually, covering a distinct feature area of the team's topic. In Deliverable 2 the team merges those feature areas into one coherent application and builds it. The product has to run, hold its own data, talk to a real external service, work in several languages, and be usable by people who do not see, hear, move or read the way the team does.

This is a team deliverable, but the grade is not purely collective. The repository records who did what and when, and the individual part of the grade is derived from that record.

1.1 Learning outcomes

- Turn a set of wireframes into a working application without quietly abandoning the design decisions they contained.
- Work as a team in a shared repository with branches, pull requests and review, producing a history that someone else can read.
- Design a data model that serves the interaction, and integrate a third-party REST API including its failure modes.
- Externalise every user-visible string and implement real internationalisation: plural rules, locale-aware formatting, text expansion, and right-to-left layout.
- Apply the universal-design and accessibility material from Week 07 to your own product and report honestly on what still fails.

1.2 What is graded, and how often

When	What happens	Graded?
Every week	Each team member commits work to the repository on a branch and opens or reviews at least one pull request	Yes — continuously, from the git history
Mid-point	Short in-class checkpoint: the application runs, data model and external API work	Yes — formative
Final	This document plus the tagged release in the repository	Yes — the main grade

A repository where all the work appears in the last week will be graded as such, regardless of how good the application is. The weekly rhythm is part of the assessed learning outcome, not an administrative formality.

2. The product you have to build

2.1 Scope

- The application implements the team's assigned topic, and it must be recognisably the product your wireframes described.
- Start from the merged screen map: every member's feature area from Deliverable 1 must be represented in the running product. If the team drops a feature area, say so explicitly in Part B, Section 3 and explain the decision — silent disappearance is treated as unfinished work.
- Target platform: web application, or mobile application, or both, consistent with the platform stated in your topic. A responsive web application is acceptable where the topic says “mobile”, provided it is designed mobile-first and demonstrated at phone width.
- Technology choice is free. Choose a stack that the whole team can actually run, and that has a mature internationalisation library — do not write your own translation layer.
- Authentication is required only where the topic needs it. If you implement it, never store passwords in plain text and never commit secrets.

2.2 Minimum functional requirements

#	Requirement	Minimum to pass
F1	Core user journey from the wireframes is implemented end to end	A user can complete the main task of your topic from start to finish without a dead end
F2	Persistent data in your own database	Data created by the user survives a restart of the application
F3	Create, read, update and delete on the main entity	All four operations reachable from the interface, with confirmation on destructive actions
F4	Integration with an external REST API related to the topic	Real data from the external service is visible in the interface and used for something, not just displayed
F5	Internationalization in at least three locales	The whole interface switches language, including dates, numbers and plurals
F6	Non-ideal states	Loading, empty, error and offline/unavailable states exist for every screen that fetches data
F7	Accessibility baseline	The ten-minute check in Section 7 passes, or every failure is documented with a plan

2.3 Definition of done

A feature counts as finished only when all of the following are true. Use this list in your pull request descriptions.

- It works for the user flow in the wireframe, not only for the happy path the developer tested.
- No user-visible string is hardcoded; every string has a translation key and exists in all locales.
- Dates, numbers, currencies and plurals go through the formatting layer, never through string concatenation.
- Loading, empty and error states are implemented.
- It is reachable and operable with the keyboard alone, and the focus indicator is visible.
- It was merged through a pull request reviewed by another team member.

3. Repository and ways of working

3.1 Repository setup

- One GitHub repository per team, created in the first week of this phase.
- Add the instructor as a collaborator (with the username “ar-rit”) in the first week, not at the end.
- Every team member commits under their own GitHub account, with their real name in the git configuration. Commits authored by a shared account cannot be attributed and will not count.
- The repository must contain a README (Section 9.2), a .gitignore, and an open-source or course license file if you choose one.
- Secrets — API keys, database passwords, tokens — are never committed. Use environment variables and commit a .env.example with empty values. A leaked key in the history costs points even if it is removed later; rotate it and say so in the README.

```
git config user.name "Ime Prezime"   &&   git config user.email
"your.university@email"
```

3.2 Branching model

Use a simple, defensible model and describe it in Part B, Section 8.1. The expected baseline:

- main is always runnable. Nobody pushes directly to main.
- develop (optional but recommended) is the integration branch.
- One branch per feature or fix, named so that the purpose is readable: feature/locale-switcher, fix/empty-state-crash, chore/ci-setup.
- Branches are short-lived: open, work, review, merge, delete. A branch that lives for six weeks is a sign the team is not integrating.
- Protect main: require a pull request and at least one approving review before merge. This is a repository setting, and we will look at whether it is switched on.

3.3 Weekly commit requirement

Who	Every team member, individually
How often	At least one meaningful commit every week of the development period
What counts	Code, tests, migrations, translation files, documentation of substance
What does not count	Whitespace-only commits, a commit that only changes the README date, committing someone else's work on their behalf, ten commits pushed in one evening to cover four empty weeks
Absence	If you cannot commit in a given week (illness, exam), say so in the weekly log in Part B, Section 9. An explained gap is treated differently from an unexplained one

3.4 Commit messages and pull requests

- Write commit messages in the imperative, describing the change, not the activity: “Add plural rules for Croatian”, not “fixes”, “update”, “work” or “asdf”.
- A conventional prefix (feat:, fix:, docs:, refactor:, test:, chore:) is recommended and makes the history readable at a glance.
- Every pull request has a description saying what changed and why, links the issue it closes, and states which definition-of-done items are met.

- Every pull request is reviewed by a team member who did not write it. A review is a comment with substance — an instant approval on a 600-line change is visible in the timeline and counts as no review at all.
- Resolve merge conflicts in your branch; never force-push over a teammate's work on a shared branch.

3.5 Issues and planning

- Use GitHub Issues for the work items, and a project board or milestones for the weekly plan.
- Each issue has an assignee. The feature areas from Deliverable 1 are the natural starting allocation.
- Link commits and pull requests to issues (Closes #12) so the history explains itself.

3.6 What we look at in the git history

We look for	What a weak history looks like
Commits spread across all weeks, from all members	One member with 80% of the commits; three members with four commits each, all in the final week
Branches that were created, worked on, reviewed and merged	Everything committed straight to main
Pull requests with real review conversation	Pull requests opened and self-merged within one minute
Readable commit messages tied to issues	“update” × 40
Steady growth of the codebase	A single initial commit containing the entire finished application
Honest history	Rewritten history and fabricated commit dates. Backdating commits is academic dishonesty, and it is trivially detectable in the push timeline

4. Database requirement

The application must own its data. A spreadsheet, a JSON file in the repository, or data held only in browser storage does not satisfy this requirement.

- Any database engine is acceptable: PostgreSQL, MySQL/MariaDB, SQLite, MongoDB, Firebase or similar. Justify the choice in one sentence in Part B, Section 4.1.
- The schema must be in the repository as migrations or as a schema file — not only inside someone's local database.
- Provide seed data so that a reviewer can run the application and see something meaningful within minutes. Seed data must be realistic and not contain real personal data of real people.
- An entity-relationship diagram is required in Part B, Section 4.2. Entities, attributes, keys and cardinalities must match what is actually in the code.
- Include at least one relationship between entities. A single flat table is below the expected level for a team project.
- Text columns that hold user-visible content must be able to store non-Latin characters — use UTF-8 (utf8mb4 on MySQL). Test with Croatian diacritics and with one non-Latin script before you claim this works.
- Store timestamps in UTC and convert for display. Store a locale preference per user if your topic has user accounts.
- No credentials in the repository; connection strings come from environment variables.

5. External REST API requirement

The application must consume at least one external REST API whose data is genuinely related to the topic. The integration has to do work in the product: enrich, validate, calculate, locate or inform. Fetching a random quote and printing it in the footer does not qualify.

5.1 Rules

- The API is called from your own backend where a key is involved, so that the key is never exposed in client-side code.
- Handle the failure modes explicitly: network error, timeout, rate limit, empty result, malformed response, and the service being down. Each one must produce a sensible interface state, not a blank screen or a raw stack trace.
- Cache responses where the data does not change every second, and respect the provider's rate limits and terms of use. Note the limits in Part B, Section 5.
- Never send personal user data to a third-party API without a reason you can defend; say in Part B what you send and why.
- If the external data is displayed to the user, attribute the source where the provider requires it.
- The API must keep working in every locale — watch out for services that return English-only text, and decide how you present it.

5.2 Candidate APIs by topic area

A starting list. Availability, limits and terms change, so verify before you commit to one, and record what you found in Part B.

API	What it gives you	Key?	Fits topics such as
Open-Meteo	Weather, forecast, historical, UV, air quality	No	Social Battery Planner, Shade & Shelter, The Interface That Disappears
Nominatim + Overpass (OpenStreetMap)	Geocoding and real-world points: benches, fountains, shops, entrances	No (strict usage policy)	Shade & Shelter, Lost in the University, Borrow Before Buying
REST Countries	Countries, languages, currencies, calling codes, formats	No	Any application with a country or locale picker
Nager.Date	Public holidays per country and year	No	Queue Without Waiting, Paper Trail, Life Admin Inbox
Frankfurter (ECB rates)	Daily and historical exchange rates	No	Cost Per Use, Repair Route, Personal Carbon Trade-Offs
HNB API (Croatian National Bank)	Official Croatian exchange rate list	No	Any product that shows money to Croatian users
Open Food Facts	Products by barcode, ingredients, packaging, nutrition	No	Expiry Radar, Object Afterlife, Cost Per Use
Wikipedia / Wikidata REST API	Articles and structured entities, multilingual labels	No	Explain My Interface, Word Keeper, First Winter
dictionaryapi.dev	Word definitions, phonetics, examples	No	Word Keeper, Plain Language Layer
OpenAlex / Crossref	Scholarly works, authors, concepts	No	Academic Uncertainty Map, Error Museum
Open Library	Books, covers, editions, subjects	No	Shared Silence, Borrow Before Buying
OpenAQ	Air-quality measurements by location	Free key	Shade & Shelter, Personal Carbon Trade-Offs
Climatiq or Carbon Interface	Emission factors and carbon estimates	Free key	Personal Carbon Trade-Offs, Object Afterlife
iFixit	Repair guides and device teardowns	No / limited	Repair Confidence, Repair Route
LibreTranslate or DeepL API Free	Machine translation	Free key	Plain Language Layer, First Winter, Word Keeper
GitHub REST API	Repositories, issues, activity	Token	The Last 10%, Error Museum, Assignment Reality Check
Transitland or a local GTFS feed	Public transport stops, routes, schedules	Free key	Micro-Decision, Lost in the University, Context Switch

If none of these fits your topic, you may use any other public API, including one you find in a directory such as publicapis.io, freepublicapis.com or the public-apis list on GitHub. Check the licence and the rate limit first, and avoid services that require a paid plan or a credit card.

6. Internationalization and localization

This section applies the Week 07 lecture, “Designing for Universal, Global & Cultural Differences”, to your own product. Internationalization (i18n) is the engineering work that makes translation and adaptation possible; localization (L10n) is the adaptation to a specific locale. Both are graded.

6.1 Required locales

Locale	Status	Why
English (en)	Required	Source language and fallback
Croatian (hr)	Required	Forces real plural rules — one / few / other — and diacritics
A third locale that stresses the system	Required — choose one	Arabic (ar) or Hebrew (he) for right-to-left; Polish (pl) or Russian (ru) for a different plural system; German (de) or Finnish (fi) for text expansion; Japanese (ja) for line breaking without spaces
A fourth locale	Optional — bonus	Only if the first three are genuinely complete

Translations may be machine-assisted, but you must check them. A machine translation that produces nonsense in the interface is still your bug. If nobody in the team speaks the third language, choose one where you can get a native speaker to look at ten screens, and say in Part B who checked it.

6.2 Engineering rules (i18n)

- Not a single user-visible string is hardcoded in a component, including button labels, placeholders, validation messages, page titles, alt text, aria-labels, e-mail templates and error pages.
- Translations live in resource files in the repository (JSON, YAML, .properties, .resx, gettext — whatever your stack uses), one file per locale, under version control.
- Keys are semantic and hierarchical: cart.checkout.button, not button_1 and not the English sentence itself used as a key.
- Never build a sentence by concatenating translated fragments; word order differs between languages. Use one message with parameters.
- No text baked into images or icons; if a graphic contains words, it needs a localised version or the text must be a text layer.
- A missing key must fall back to a defined chain (hr → en) and must be visible in development, never shown to a user as a raw key.
- The HTML lang attribute changes with the locale, and dir changes for right-to-left locales. This matters for screen readers, hyphenation and search engines.
- The chosen locale is persisted (URL segment, cookie or user profile) and survives a reload. Do not infer the language from the IP address alone — a Croatian student in Zagreb may well want the interface in English.

```
// one message, parameters inside — never "You have " + n + " items"
inbox.unread = "{count, plural, one {# nepročitana poruka} few {# nepročitane poruke} other {# nepročitanih poruka}}"
```

6.3 Localization rules (L10n)

- Plurals go through the locale's real plural rules. Croatian has three forms and Arabic has six; an if (n === 1) is wrong in both.
- Dates, times, numbers, percentages and currencies are formatted by the platform's internationalisation API (Intl.DateTimeFormat, Intl.NumberFormat, Intl.PluralRules, Intl.RelativeTimeFormat, Intl.ListFormat) or the equivalent in your stack — never with manual string building.
- Sorting of names and lists uses a locale-aware collator, not byte order. Č and Ć do not sort where ASCII puts them.
- Decimal and thousands separators, first day of the week, 12- versus 24-hour clock, and currency symbol position all follow the locale, not your habits.
- Timestamps are stored in UTC and rendered in the user's time zone.
- Address, postcode and phone fields accept international formats; a validation rule that demands a five-digit postcode excludes most of the world.
- Name fields: a single full-name field, generously sized, is safer than first/middle/last. Do not require a surname, do not reject apostrophes, hyphens, spaces or non-Latin characters, and do not force capitalisation.

6.4 Layout rules

- Text expansion: translations from English can grow by roughly a third, and short labels can grow much more. No fixed-width buttons, no single-line assumptions, no truncation that hides meaning. Test the layout with the longest locale.
- Right-to-left: if your third locale is Arabic or Hebrew, the layout mirrors. Use CSS logical properties (margin-inline-start, padding-inline-end, text-align: start) rather than left and right, set dir on the document, mirror directional icons such as back arrows, and leave clocks, media controls and checkmarks unmirrored. Numbers and Latin product names stay left-to-right inside right-to-left text.
- Line breaking differs: Japanese and Chinese break without spaces, German produces very long compounds. Do not rely on a specific wrap point.
- Fonts must actually contain the glyphs of all your locales, including diacritics. Check č, ć, đ, š, ž render correctly in every weight you use.

6.5 Cultural rules

- Colour meaning is not universal — check that any colour carrying meaning in your product (status, gain and loss, warning) is not the only carrier of that meaning, and is not actively misleading in another culture.
- Emoji and icons: gestures, animals and symbols carry different meanings across cultures, and an icon that is obvious to you may be meaningless elsewhere. Pair icons with text labels.
- Imagery and examples: default example data, placeholder names and sample avatars reveal who you imagined your user to be. Make them plural.
- If your product uses AI-generated content, be aware that models carry cultural bias and tend to produce Western-default answers; state in Part B how you handled this if it applies to your topic.

7. Accessibility and universal design

The target is WCAG 2.2 level AA. You are not expected to produce a fully conformant product in one semester; you are expected to design for it deliberately, test it, fix what you can, and report honestly on what remains.

7.1 Why this is not optional

Accessibility is a legal requirement for a large part of the digital market in Europe, and the industry baseline is poor — the overwhelming majority of home pages still fail automated checks, with low contrast, missing alternative text and unlabelled form fields at the top of the list every year. Those three failures are also the easiest ones to avoid in a project of this size, which is why they are explicitly graded here.

7.2 Required practices

- Semantic HTML first: real buttons, real headings in order, real lists, real form labels. A div with a click handler is not a button and will fail the keyboard check.
- Every image has an alt attribute: descriptive when the image carries meaning, empty when it is decorative.
- Every form control has a programmatically associated label, and errors are described in text, identified clearly, and linked to the field.
- Colour contrast at least 4.5:1 for normal text and 3:1 for large text and interface components.
- Colour is never the only way information is conveyed — pair it with text, shape or an icon.
- Full keyboard operation with a visible focus indicator and a logical tab order; no keyboard traps.
- Interactive targets at least 24 by 24 CSS pixels, with spacing; 44 pixels is the comfortable size on touch.
- The interface survives 200% zoom and a 320-pixel-wide viewport without loss of content or function.
- Respect prefers-reduced-motion; nothing flashes more than three times per second.
- Accessible names on icon-only buttons, and live regions for content that updates without a page load.

7.3 The ten-minute check

Run this on the finished product, record the result in Part B, Section 7, and fix what you can.

#	Check	How
1	Unplug the mouse	Reach and operate every feature with Tab, Shift+Tab, Enter, Space and arrow keys
2	Focus visible	Is it always obvious where you are? Never remove the outline without replacing it
3	Zoom to 200%	Does anything disappear, overlap or stop working?
4	Automated scan	Run an automated checker on your main pages and record the issues, remembering it catches only a fraction of problems
5	Contrast	Check the real colours of text, buttons, disabled states and placeholder text
6	Images	Does every image have a sensible alt, and are decorative ones empty?

#	Check	How
7	Forms	Click each label — does it focus its field? Submit an empty form — is the error understandable?
8	Headings and language	Is the heading order sensible, and does the lang attribute change with the locale?
9	Screen reader	Complete one full task with a screen reader (NVDA, VoiceOver, TalkBack or Narrator) and write down where you got lost
10	Targets and motion	Are controls big enough and spaced, and does the interface calm down with reduced motion enabled?

8. Weekly rhythm

Dates are filled in by the instructor. The milestones are the expected state of the repository at the end of each week; a team that is ahead may reorder them, but every week must show work from every member.

Week	Dates	Expected state of the repository	Done
1		Repository created, instructor added, README skeleton, project runs locally for everyone, branching model agreed and written down	
1		Merged screen map agreed; data model drafted; first migrations; issues created and assigned	
1		First vertical slice: one screen reading and writing real data through the database	
2		External REST API integrated behind a service layer, with error and empty states	
2		i18n infrastructure in place: all strings externalised, locale switcher working, English complete	
2		Croatian complete, including plurals and date/number formatting	
2		Third locale complete; text expansion and right-to-left (or line-breaking) checked	
3		Feature complete against the minimum requirements; non-ideal states everywhere	
3		Accessibility pass: ten-minute check run, issues logged as GitHub issues, the cheap ones fixed	
3		Testing, bug fixing, seed data, deployment or run instructions verified on a clean machine	
4		Documentation, this document completed, release tagged, demo rehearsed	
4		Final submission and presentation	

9. Submission

9.1 What to submit

This document	Completed, exported to PDF, one per team
Repository link	Public, or private with the instructor added as a collaborator
Release tag	Tag the submitted state, for example v1.0-deliverable2, so we grade the same code you demonstrated
Run instructions	In the README: prerequisites, environment variables, how to seed the database, how to start
Demo video (optional)	A short walkthrough is welcome but does not replace a running application
Deployment (optional)	A live URL is a bonus, not a requirement; a free tier is enough

9.2 README requirements

- What the application does, in three sentences, and which topic it implements.
- The team, with GitHub usernames and the feature area each person owned.
- Technology stack and prerequisites with versions.

- Setup: install, environment variables (with .env.example), database creation, migrations, seed data, start command.
- Which external API is used and how to obtain a key, if one is needed.
- Supported locales and how to add a new one.
- Known limitations, including the accessibility issues you did not manage to fix.

9.3 Assessment

Criterion	Points	What is being judged
Working application against the wireframes	20	The core journey runs; the product is recognisably the one that was designed; non-ideal states exist
Git history and teamwork	20	Weekly commits from every member, branches, reviewed pull requests, readable history, issues used
Internationalisation and localisation	20	Three locales complete, no hardcoded strings, real plural rules, locale-aware formatting, expansion and direction handled
Accessibility and universal design	15	Ten-minute check performed and reported, semantic markup, contrast, keyboard operation, honest list of remaining issues
Database design and implementation	10	Model fits the interaction, migrations and seed data in the repository, diagram matches the code
External REST API integration	10	Relevant to the topic, used for something, failure modes handled, keys kept out of the repository
Documentation and submission quality	5	README lets a stranger run the project; this document is complete and matches the repository
Total	100	

Individual adjustment: the team grade is the starting point for every member, and is then adjusted up or down according to the contribution visible in the git history, the pull-request record and the weekly log. A member with no traceable contribution cannot pass on the strength of the team's work.

9.4 Academic integrity and AI use

- AI assistants may be used for code, and most of you will use them. Declare them in Part B, Section 12, and be able to explain any line of code in your repository during the presentation.
- Code you did not write and do not understand is the single most reliable way to fail the oral part of this deliverable.
- Generated code that introduces an accessibility or internationalisation defect is your responsibility — that is precisely what this deliverable teaches you to catch.
- Attribute third-party code, snippets and libraries in the README. Using libraries is expected and good; passing them off as your own work is not.
- Fabricated, backdated or rewritten git history to simulate weekly work is treated as academic dishonesty.

PART B — SUBMISSION TEMPLATE (fill this in)

1. Team, product and repository

1.1 Identification

Team name / number	Team 2
Project topic (number and title)	Personal Carbon Trade-Offs (the topic number is not recorded in the team's course materials, so only the title is given)
Application name	Carbon Choices
Platform (web / mobile / both)	Web (responsive, designed mobile-first; demonstrated at 320–375 px phone width)
Repository URL	https://github.com/fa6375/personal-carbon-tradeoffs
Release tag submitted	v1.0-deliverable2 (annotated tag on the final main commit)
Live deployment URL (if any)	None (runs locally; see README → Getting Started and "npm run demo")
Date of submission	13 November 2026

1.2 Team members

Name	GitHub username	Feature area owned (from Deliverable 1)	Main responsibility in the build
Faraj Aliyev	fa6375	Compare Everyday Choices (10 screens)	Frontend foundation and app shell (PR #1), Compare feature (PR #4), shared backend: Firebase Authentication, Cloud Firestore, security rules, CRUD repositories, OpenStreetMap API, data import (PR #7), final fixes, demo data and report assembly (PR #9)
Martin Mudrinić	martinmudrinic	Carbon Reduction Goals & Recommendations (10 screens)	Goals feature (PRs #2, #5), localization en/hr/de, accessibility and loading/ empty/error/offline states for the whole app (PR #8)
Lazar Jovović	lj7829	Personal Carbon Tracking (10 screens)	Tracking feature (PRs #3, #6); documentation, README and evidence: responsible for reviewing the final report, README and evidence in PR #9 before submission

1.3 The product in three sentences

Carbon Choices helps students and other everyday travellers see the carbon cost of daily choices next to what they cost in money and time, so trade-offs are visible instead of hidden. Users compare ways of making the same journey (with real route distances from OpenStreetMap), record their transport, food and home-energy activities, and turn the result into a monthly carbon budget with a plan they can follow. Unlike single-number carbon calculators, it weighs CO_2 against cost and travel time using the user's own priorities, works in English, Croatian and German, and keeps every user's data private in their own account.

1.4 Elevator demo path

The exact sequence of screens a reviewer should follow to see the product work. Be specific — this is what will be clicked during grading.

Step	Screen	What the reviewer should see	Which requirement it demonstrates
1	Terminal: npm run demo	Emulators start, demo data is loaded, a demo e-mail and password are printed	F2 seed data, run instructions
2	Account (#/account) → Sign in	Signed in as demo@carbon-choices.test; "Data saved in this browser" panel	Authentication
3	Carbon tracking → Dashboard	This month: totals, daily average, largest source, recent activities	F1, F2
4	Add activity → Transport → "Find the distance on a map"	Enter two places (e.g. Trg bana Jelačića, Zagreb → Maksimir, Zagreb), Look up distance → car/bike/foot distances from OpenStreetMap; use one and save	F4, F3 (create)
5	History → open the activity → Edit → Delete (confirm)	Edited value shown; delete asks for confirmation; reload → data kept	F3, F2
6	Compare journeys → Start → choose 4 options → journey (with map lookup) → priorities → results	Cards, bars, differences table, recommendation with reasons	F1, F4
7	Breakdown / What if / Save tabs → Saved comparisons	Formulas with real numbers; scenario recalculated; saved list with Open/ Delete	F1, F3
8	Carbon goals → steps 1–5	Budget from tracked data, monthly plan, progress against this month's target, adjustment history; "Your plan is saved in your account"	F1, F2
9	Language selector → Hrvatski, then Deutsch; reload	Whole interface, numbers (3,05 €), dates and plurals (3 aktivnosti) switch; choice kept after reload	F5

Step	Screen	What the reviewer should see	Which requirement it demonstrates
10	Narrow the window to 320 px; Tab through Compare	No horizontal scrolling; visible focus outline on every control	F7

2. Architecture and technology

2.1 Stack

Layer	Technology and version	Why this choice
Frontend	React 19.3.1 + TypeScript 6.0.3, built with Vite 8.3.7; plain CSS with design tokens	Mature, well-documented stack the whole team could run; TypeScript catches missing translation keys and data-shape errors at build time; Vite gives fast reloads
Backend / API	No custom server. Firebase is the managed backend: Firebase Authentication + Cloud Firestore accessed with the Firebase JS SDK 12.19.0; access control in firestore.rules	No backend code to host or secure; Firestore Security Rules enforce per-user ownership and validation; the chosen external API needs no secret key
Database	Cloud Firestore (Native mode), project personal-carbon-tradeoff, database (default)	Document model fits per-user data (activities, comparisons, one plan); real-time listener keeps Tracking and Goals in sync; free tier is enough
i18n library	Custom typed module in src/i18n (no external library) using Intl.PluralRules, Intl.NumberFormat, Intl.DateTimeFormat, Intl.Collator	Deviation from brief §2.1: see §6.2 for the reason. 903 keys per locale, checked by TypeScript and tests
Testing	Vitest 4.1.12, jsdom 26.1.0, Testing Library 16.3.3, axe-core 4.14.0, @firebase/rules-unit-testing 5.0.2, Firebase Emulator Suite (firebase-tools 15.36.0)	Unit, component, accessibility and security-rules tests run locally without touching real data
CI / automation (if any)	None (no CI); GitHub ruleset "Protect main" requires a pull request with one approval	Checks are run locally: npm run build, lint, test, test:emulator
Hosting (if any)	None (not deployed). Runs locally; firestore.rules deployed to Firebase with npm run deploy:rules	Deployment is optional in the brief

2.2 Architecture diagram

Components, where data flows, and where the external API sits. Hand-drawn and photographed is acceptable if it is legible.

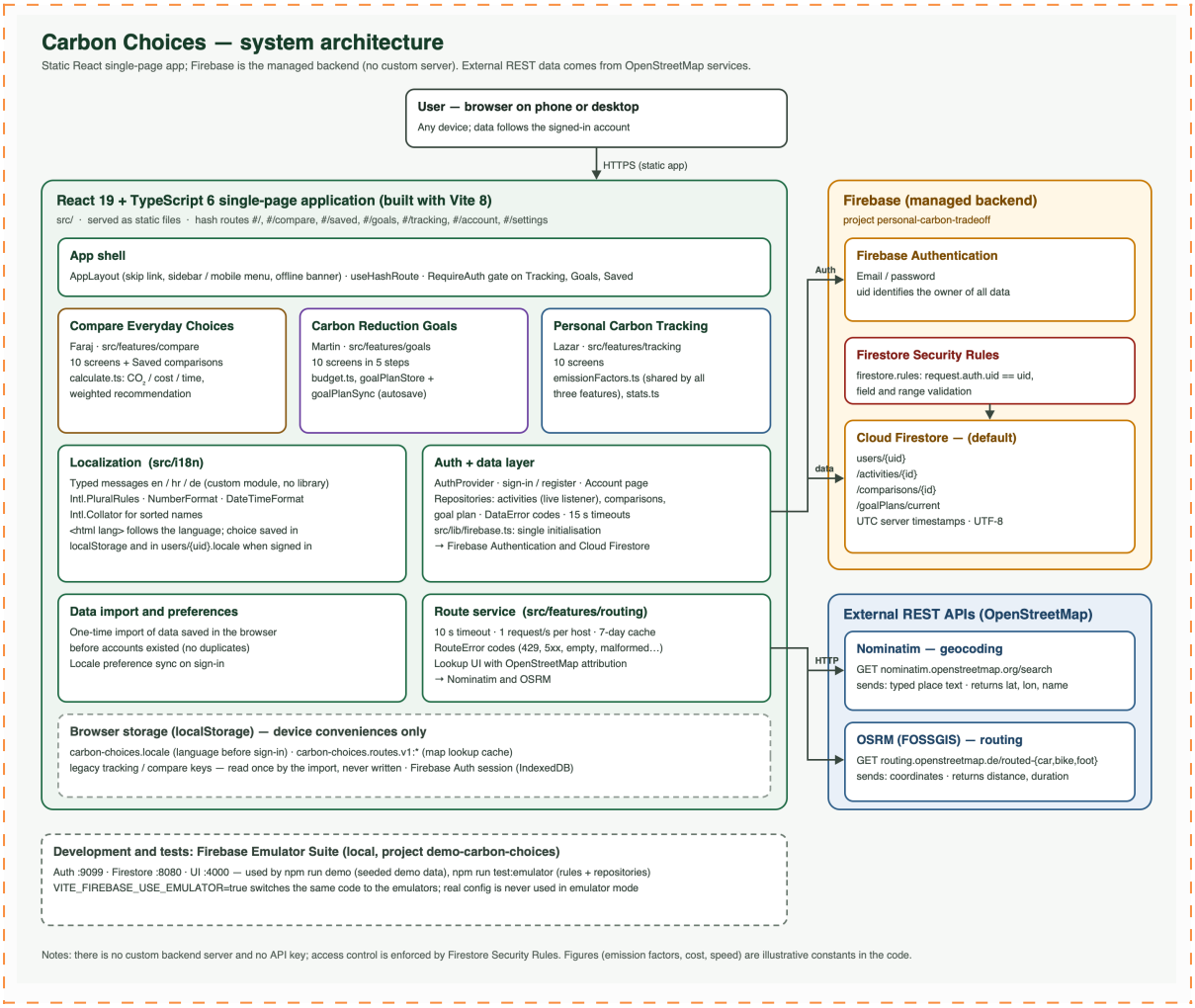


Figure 2.1 — system architecture

2.3 Repository structure

The top-level folders and what lives in each. Keep it to the level a new team member would need.

src/ — the application. components/ (shared UI: buttons, dialogs, navigation), layouts/ (app shell), pages/ (one page per route), features/compare, features/goals, features/tracking (one folder per Deliverable 1 feature area: screens, calculations, Firestore repositories), features/auth, features/account, features/routing (sign-in, data import and locale preference, OpenStreetMap service), i18n/ (translation module and locales/en|hr|de), lib/ (Firebase initialisation, error codes), styles/ (design tokens), test/ (component and accessibility tests).

tests/emulator/ — security-rules and repository tests against the Firebase emulators. **tests/live/** — opt-in test against the real project.
scripts/ — seed-demo.mjs + seedDemo.ts: demo data for the emulators (npm run demo).
docs/ — this report, the requirements audit, diagrams/ (ERD and architecture: editable .svg source, generator scripts and .png) and evidence/ (screenshots and test logs).
Root — firestore.rules, firebase.json, .firebaserc, .env.example, .env.emulator, package.json, vite/vitest configs, README.md.

3. From wireframes to screens

3.1 Merged screen map

Every screen in the running application, traced back to the wireframe it came from and the person who designed it.

#	Screen in the application	From whose Deliverable 1 set	Wireframe no.	Changed since the wireframe? How?
1	Compare: Start	Faraj	C1	As designed
2	Compare: Choose transport	Faraj	C2	As designed (6 options, at least 2)
3	Compare: Enter journey	Faraj	C3	Changed: optional OpenStreetMap distance lookup added; typing a distance still works
4	Compare: Set priorities	Faraj	C4	As designed (0–3 for CO_2, cost, time)
5	Compare: Comparison results	Faraj	C5	As designed; cards, bars and a differences table
6	Compare: Detailed breakdown	Faraj	C6	As designed; shown as a result tab
7	Compare: Recommended option	Faraj	C7	As designed; shown as a result tab
8	Compare: Alternative scenario ("What if")	Faraj	C8	As designed; shown as a result tab
9	Compare: Save comparison	Faraj	C9	Changed: needs sign-in; saved list moved to its own page
10	Compare: Error screen	Faraj	C10	As designed (missing/invalid input, failed calculation, not found, storage)
11	Goals step 1: Carbon budget setup	Martin	G1	Combined with G2 on one step page
12	Goals step 1: Personal carbon budget	Martin	G2	Combined with G1
13	Goals step 2: Budget allocation by category	Martin	G3	Combined with G4; changed: 3 categories (goods & shopping dropped)
14	Goals step 2: Trade-off priorities	Martin	G4	Combined with G3
15	Goals step 3: Monthly carbon plan	Martin	G5	Combined with G7
16	Goals step 4: Budget adjustment	Martin	G6	Combined with G8 and G9

#	Screen in the application	From whose Deliverable 1 set	Wireframe no.	Changed since the wireframe? How?
17	Goals step 3: Reduction opportunities	Martin	G7	Combined with G5; example ideas with calculated savings
18	Goals step 4: Goal progress	Martin	G8	Changed: uses real tracked activities
19	Goals step 4: Budget adjustment details	Martin	G9	Combined with G6 and G8
20	Goals step 5: Carbon budget summary	Martin	G10	As designed
21	Tracking: Dashboard	Lazar	T1	As designed (tab)
22	Tracking: Add activity	Lazar	T2	As designed
23	Tracking: Transport activity entry	Lazar	T3	Changed: optional map distance lookup; shares one form component with T4/T5
24	Tracking: Food activity entry	Lazar	T4	Shares the form component
25	Tracking: Home energy entry	Lazar	T5	Shares the form component
26	Tracking: Activity history	Lazar	T6	As designed, with search and filters (tab)
27	Tracking: Activity details	Lazar	T7	As designed
28	Tracking: Edit activity	Lazar	T8	As designed; delete with confirmation dialog
29	Tracking: Carbon emissions breakdown	Lazar	T9	As designed (tab)
30	Tracking: No activities recorded	Lazar	T10	Shown as the empty state of Dashboard, History and Breakdown
31	Home (#/)	— (new)	—	New: entry page shared by all three areas
32	Saved comparisons (#/saved)	Faraj (from C9)	C9	New page: list, open and delete saved comparisons
33	Account (#/account) and sign-in gate	— (new)	—	New: sign in / register / sign out, import of data saved in the browser
34	Settings (#/settings)	— (new)	—	New: language selection
35	Offline banner, Not found page	— (new)	—	New: non-ideal states for the whole app

3.2 What was dropped or merged, and why

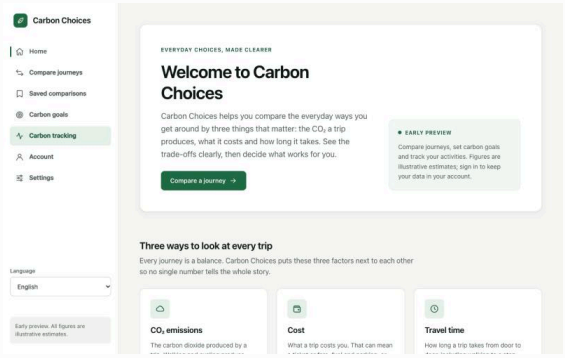
No feature area was dropped: all 30 Deliverable 1 screens are reachable in the running app.

Merged: Martin's ten Goals screens are grouped into five steps (Set up, Allocate, Plan, Track, Summary) so a user can move through the plan in order on a phone without ten separate pages; every step can still be opened directly. Lazar's three entry screens (T3–T5) share one form component with category-specific fields. Compare screens 6–9 are tabs of the results screen.
Changed: the goods & shopping category was dropped from Goals so that Goals uses exactly the Tracking categories (transport, food, home energy) and can read real tracked data. Saving needs an account (Firebase Authentication), so sign-in and an Account page were added. An optional OpenStreetMap distance lookup was added to Compare and Tracking.
Traceability: the Deliverable 1 wireframes are kept in each member's individual Deliverable 1 submission, not in the repository; the screen map above traces every implemented screen to its wireframe by number and name.

3.3 Screens of the finished application

Add as many pairs as you need. Show the application as it actually looks, at the size a user would see it.

Screen: Home



Screen: Compare: start

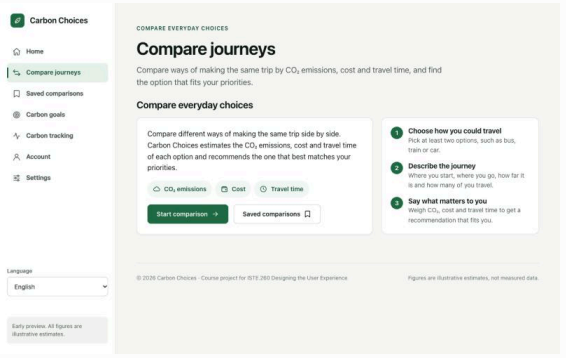
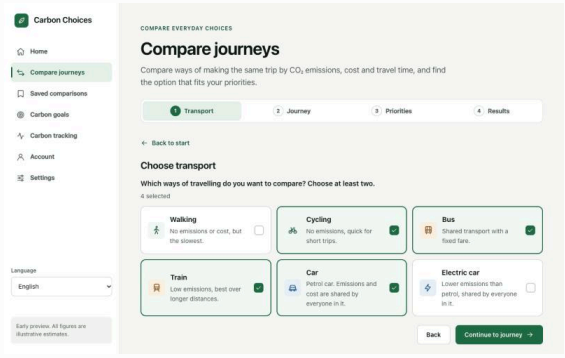


Figure 3.1 — implemented screens

Screen: Compare: choose transport



Screen: Compare: journey + map distance

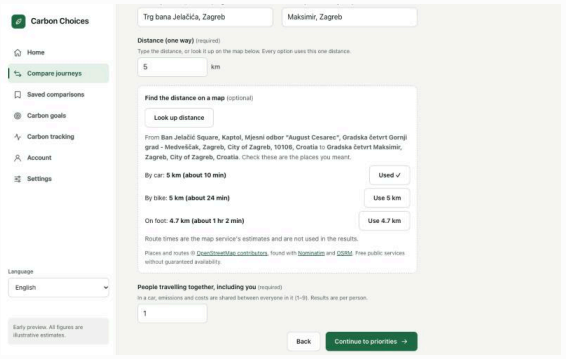


Figure 3.2 — implemented screens

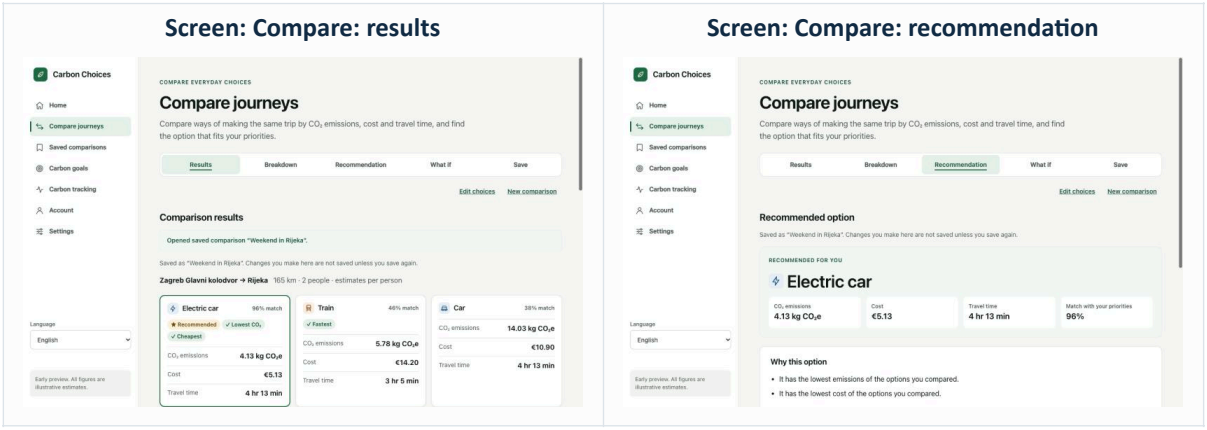


Figure 3.3 — implemented screens

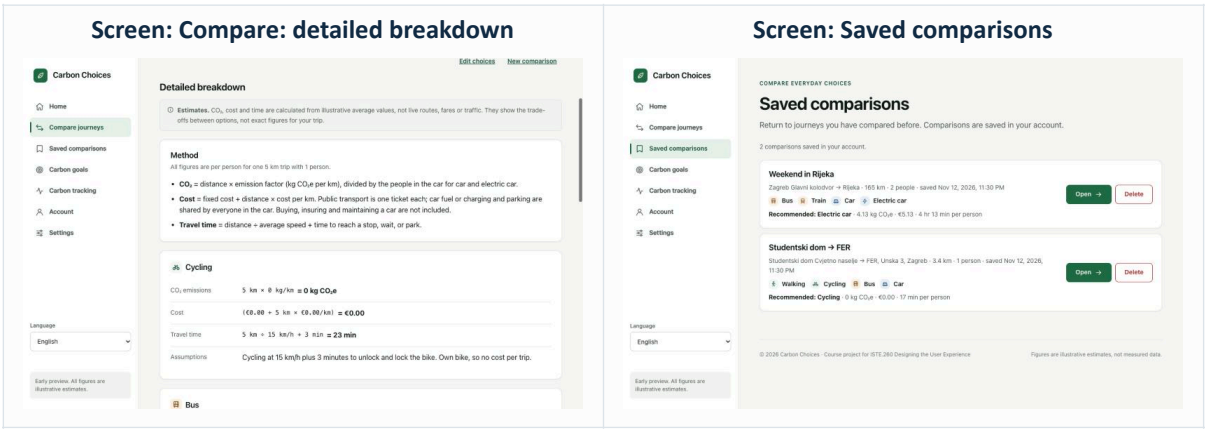


Figure 3.4 — implemented screens

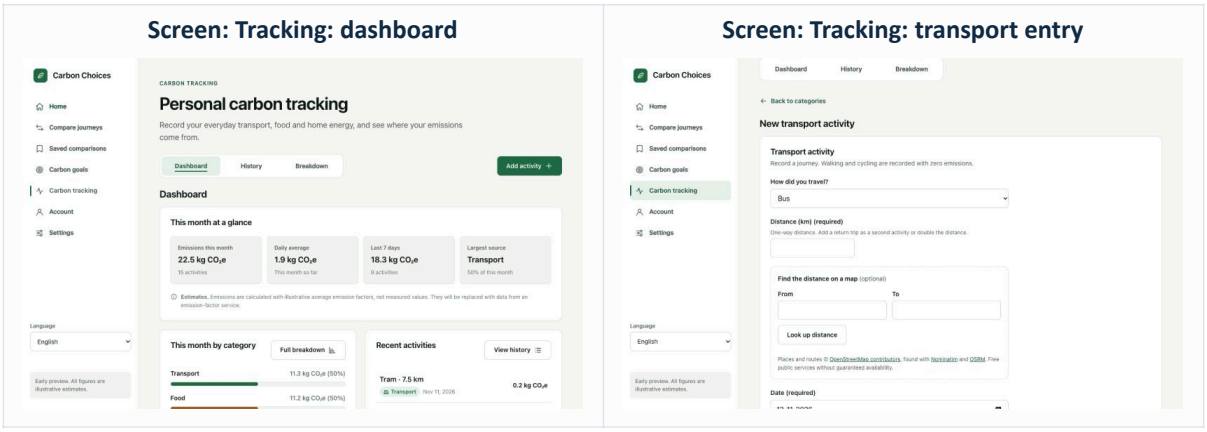


Figure 3.5 — implemented screens

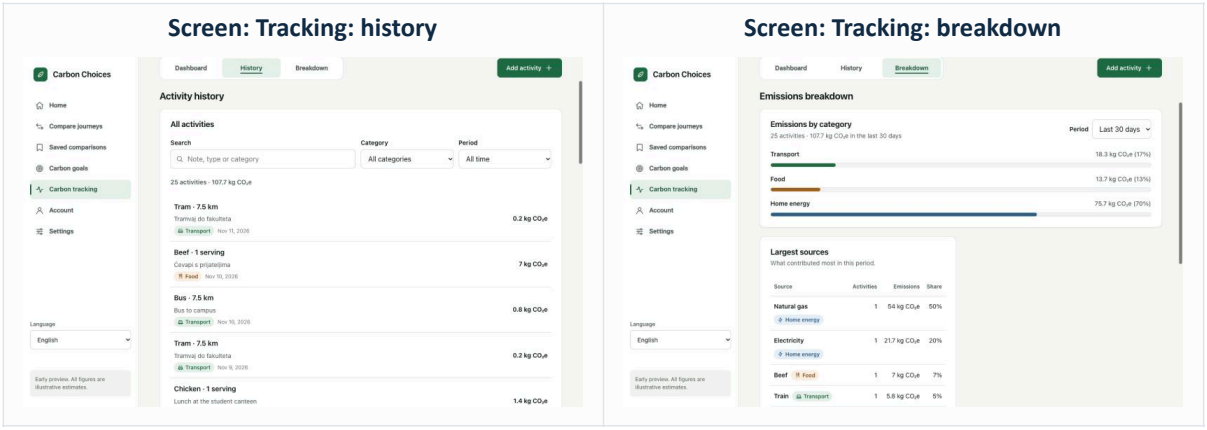


Figure 3.6 — implemented screens

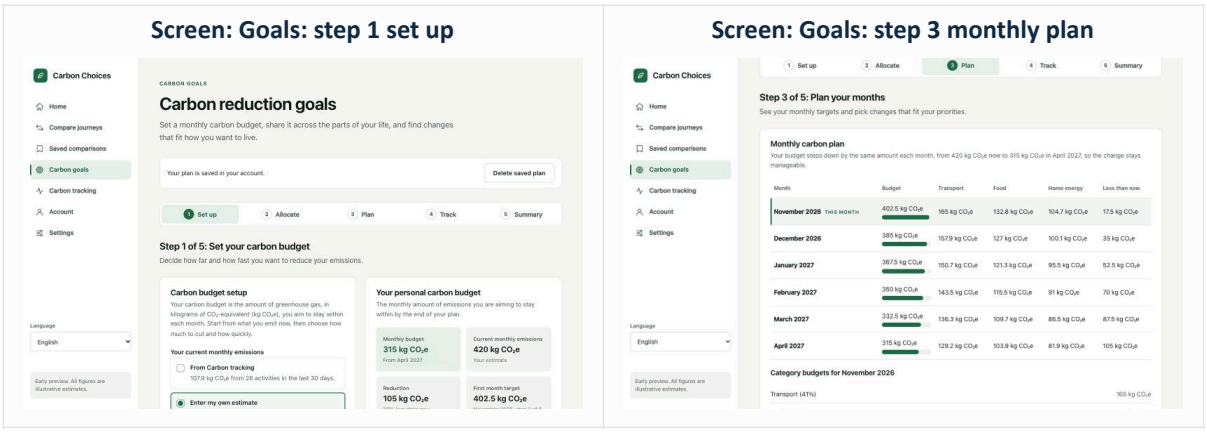


Figure 3.7 — implemented screens

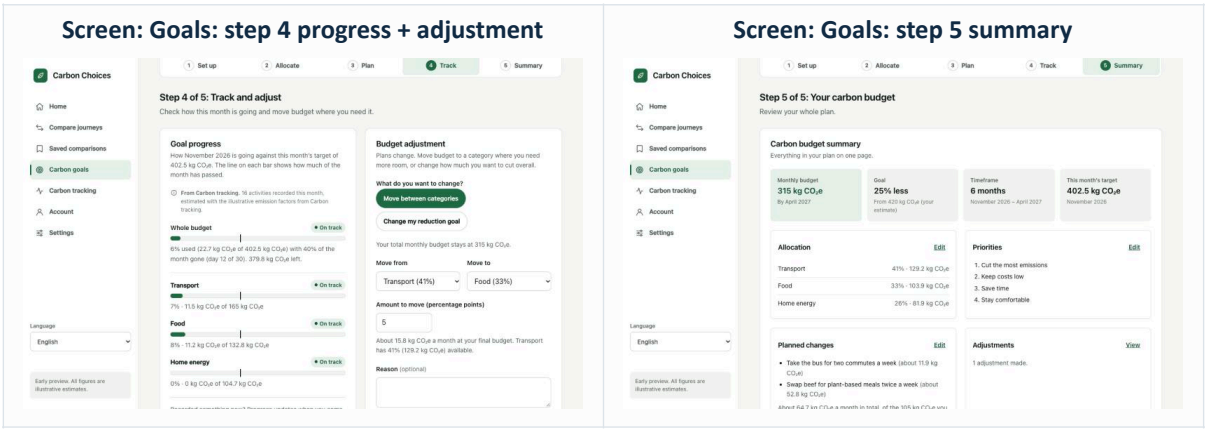


Figure 3.8 — implemented screens

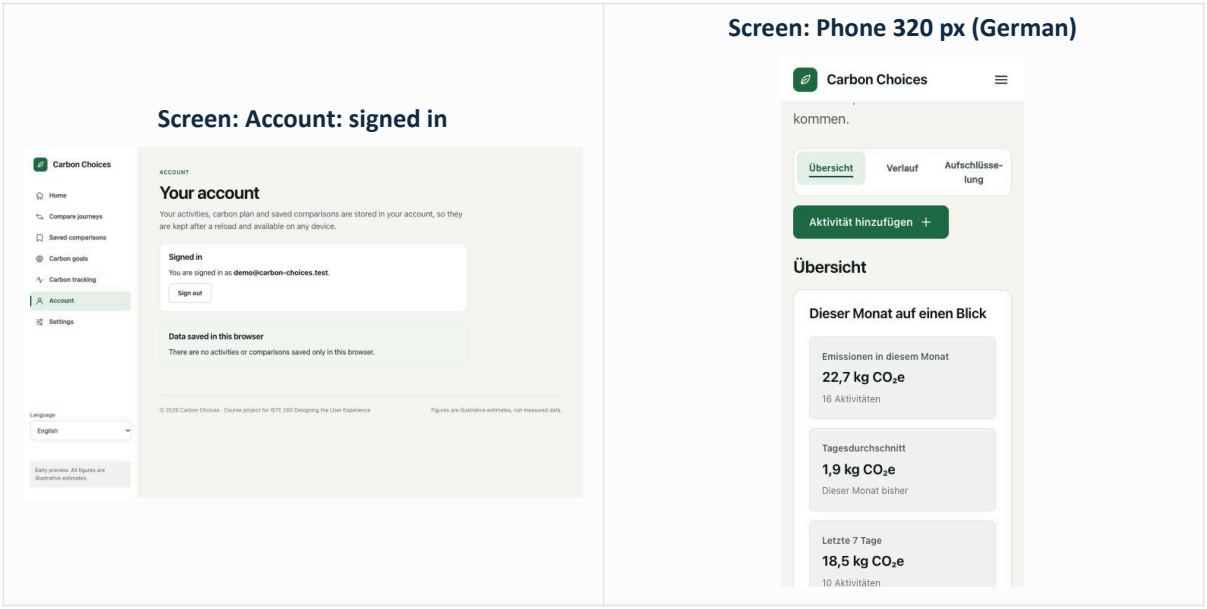


Figure 3.9 — implemented screens

4. Database

4.1 Engine and justification

Database engine and version	Google Cloud Firestore (Native mode, Firebase JS SDK 12.19.0), database (default) in project personal-carbon-tradeoff
Why this engine	A managed document database with per-user Security Rules suits private per-user data and needs no server for the team to host.
Where the schema lives in the repository	Firestore is schemaless: the schema is enforced by firestore.rules (field names, types, ranges) and described by the TypeScript types in src/features/*/types.ts; documented in README → Database and the ERD below
Migration tool / approach	No migration tool: Firestore needs no table migrations. A schemaVersion field on goal plans allows future data changes; rules are versioned in Git and published with npm run deploy:rules
How to load the seed data	npm run demo: starts the Firebase emulators, runs scripts/seed-demo.mjs (25 activities, 2 saved comparisons, 1 plan for demo@carbon-choices.test; fictional data) and starts the app. Nothing is written to the real project.
Character set / collation	UTF-8 (Firestore strings are Unicode). Tested with Croatian (č ć đ š ž), Cyrillic and Arabic text stored and read back (tests/emulator/repositories.test.ts). Timestamps are Firestore server timestamps in UTC, shown in the user's locale.

4.2 Entity-relationship diagram

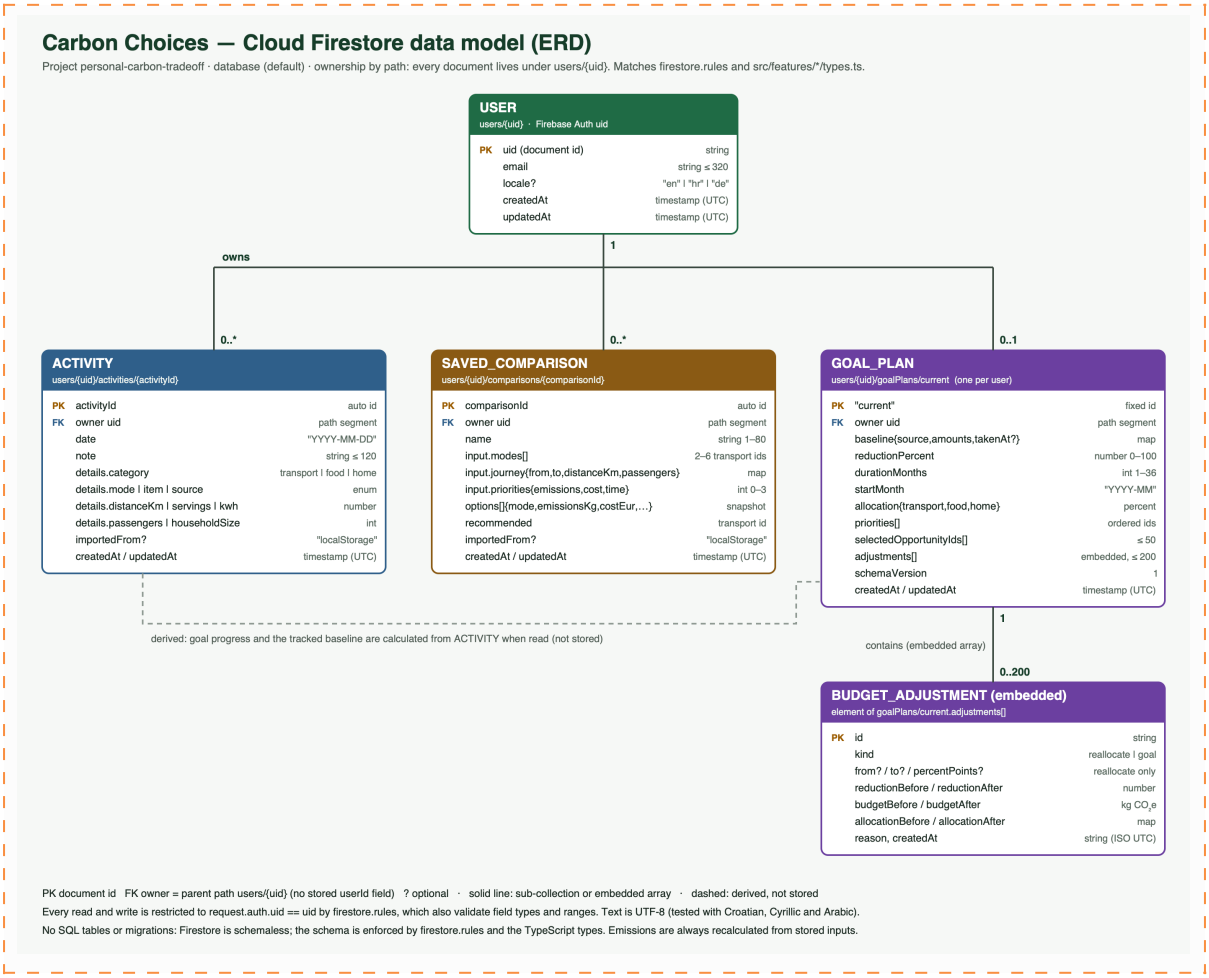


Figure 4.1 — entity-relationship diagram

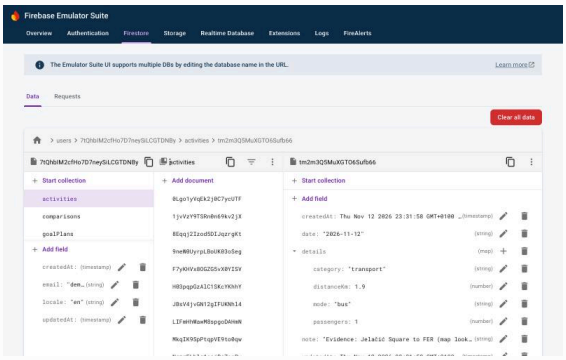
4.3 Entities

Entity / table	What it holds	Key relationships	Notes (indexes, constraints, soft delete …)
USER — users/{uid}	E-mail, optional interface language (locale), created/ updated time	Root of all user data; uid = Firebase Auth uid	Created on first sign-in; rules allow only the owner; locale ∈ {en, hr, de}
ACTIVITY — users/{uid}/ activities/{id}	One tracked activity: date, note, category-specific details	Many per USER (sub-collection)	Queried ordered by date; emissions calculated, not stored; ranges validated in rules
SAVED_COMPARISON — users/{uid}/comparisons/{id}	A saved Compare result: name, input (modes, journey, priorities), options snapshot, recommendation	Many per USER	Cannot be edited after saving (rules allow create/read/ delete only)

Entity / table	What it holds	Key relationships	Notes (indexes, constraints, soft delete …)
GOAL_PLAN — users/{uid}/goalPlans/current	The user's carbon plan: baseline, goal, duration, allocation, priorities, chosen ideas	0..1 per USER (fixed id "current")	Autosaved; createdAt fixed by rules; schemaVersion 1
BUDGET_ADJUSTMENT (embedded)	History of budget moves and goal changes with before/after values	0..200 inside GOAL_PLAN.adjustments[]	Embedded because always read with the plan

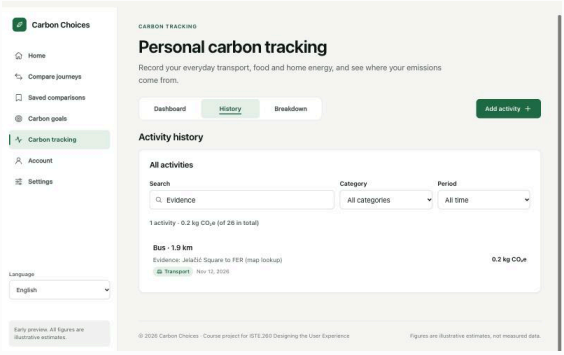
4.4 Evidence that it works

In the database



Emulator UI: users/{uid}/activities/{id} (note "Evidence: Jelačić Square to FER")

In the application



Same activity in History after a page reload

Figure 4.2 — persistence evidence

Describe one non-trivial query or operation in the application and why it is shaped that way.

Importing data saved in the browser without duplicates (src/features/account/localImport.ts). Before accounts existed, activities and comparisons were saved in localStorage. On request, the import first reads all existing document ids in users/{uid}/activities and /comparisons from the server, then writes each local record with its local id as the Firestore document id in batches of at most 450 writes, skipping ids that already exist. Using the record id as the document id makes the import idempotent: running it twice creates no duplicates, and a failed batch can simply be retried. Original timestamps are kept, which is why the rules accept createdAt values in the past but never in the future. Covered by the emulator test "imports once and skips duplicates on a second run".

5. External REST API integration

5.1 The service

API name and provider	Nominatim (geocoding) and OSRM (routing), both OpenStreetMap services; OSRM run by FOSSGIS e.V.
Documentation URL	https://nominatim.org/release-docs/latest/api/Search/ · https://project-osrm.org/docs/v5.24.0/api/
Why it is relevant to our topic	The distance of a journey drives every CO_2, cost and time figure in Compare and every transport activity in Tracking; users often do not know it. Real route distances make the carbon comparison meaningful.
Authentication (none / key / token) and where the secret lives	None — both services are free and need no key, so they are called from the browser; there is no secret to protect.
Rate limits and terms we must respect	Nominatim: max. 1 request per second, no autocomplete, cache results, identify the application (browser Referer; optional contact e-mail VITE_NOMINATIM_EMAIL). FOSSGIS OSRM: fair use, no heavy use. Both are public services without guaranteed availability.
Attribution required by the provider?	Yes: "© OpenStreetMap contributors" (ODbL) with a link, shown with every lookup, plus links to Nominatim and OSRM.
Where the integration lives in the code (file / module)	src/features/routing/routeService.ts (requests, throttle, cache, errors), routeMessages.ts (translated errors), RouteDistanceLookup.tsx (UI)

5.2 Endpoints used

Endpoint	Method	What we send	What we use from the response	Where it appears in the UI
nominatim.openstreetmap.org/search?q=…&format=jsonv2&limit=1	GET	The place text the user typed (≤ 200 characters); no account data	lat, lon, display_name of the first match	Compare → Enter journey; Tracking → transport form ("Find the distance on a map")

Endpoint	Method	What we send	What we use from the response	Where it appears in the UI
routing.openstreetmap.de/routed-car/route/v1/driving/{lon},{lat};{lon},{lat}?overview=false	GET	The two coordinates only	routes[0].distance (m) and duration (s)	"By car" distance; the user can apply it to the distance field
routing.openstreetmap.de/routed-bike/route/v1/driving/…	GET	Coordinates	distance, duration	"By bike" distance
routing.openstreetmap.de/routed-foot/route/v1/driving/…	GET	Coordinates	distance, duration	"On foot" distance

5.3 Failure handling

Failure mode	What the user sees	What the system does
Network unavailable / offline	A translated message: the map service could not be reached; check the connection or type the distance. When the browser is offline the lookup is not attempted and the offline banner is shown.	RouteError "network"; typed distance stays usable
Request timeout	"The map service took too long to answer…"	Request aborted after 10 s; RouteError "timeout"
Rate limit reached (429)	"The free map service is busy (too many requests). Wait a minute…"	HTTP 429 → "rate-limited"; requests are throttled to 1/s per host to avoid it
Empty result set	"The destination could not be found on the map. Try adding the town or country…" (screenshot below)	Empty Nominatim result → "not-found" naming the start or destination; OSRM NoRoute → profile shown as "no route found"
Malformed or unexpected response	"The map service sent an answer the app could not understand…"	JSON is validated (array, numeric lat/lon in range, route distance ≥ 0) → "malformed"
Service down (5xx)	"The map service is having problems right now…"	HTTP 5xx → "server-error"

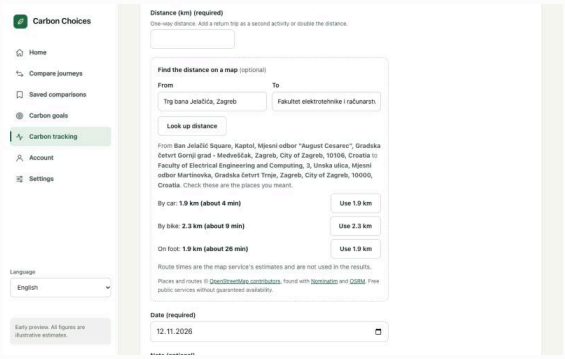
Failure mode	What the user sees	What the system does
Invalid or expired key	"The map service refused the request. The service settings may be wrong."	HTTP 401/403 → "unauthorized" (no key is used today; relevant if a keyed provider is configured)

5.4 Caching and data volume

Geocoding results are cached per normalised query text and route results per profile and coordinate pair, in memory and in localStorage (carbon-choices.routes.v1:*), for 7 days. Places and road networks change slowly, so a week-old distance is still accurate for an estimate, and the cache keeps repeated lookups (for example a daily commute) from re-using the free public services. A typical lookup makes 5 requests (2 geocoding + 3 routes) the first time and 0 when repeated; requests to each host are spaced at least 1.1 s apart, so one lookup takes about 3 seconds. All of this is covered by unit tests (src/features/routing/routeService.test.ts).

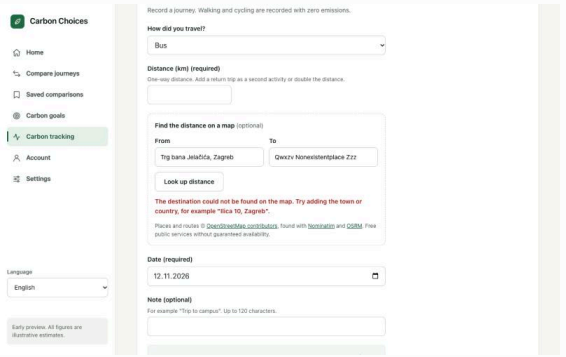
5.5 Evidence

Working state



Real distances: Jelačić Square → FER, Zagreb

Failure state



Empty result: destination not found

Figure 5.1 — external API in use, and when it is not available

6. Internationalization and localization

6.1 Locales implemented

Locale	Language	Completeness (%)	Who produced it	Who checked it	Why this locale
en	English	100 % (903 of 903 keys)	Team (source language)	Team	Source language and fallback
hr	Croatian	100 % (903 of 903 keys)	Martin Mudrinić (PR #8); month grammar fixed in PR #9	Checked by the team in the running app (plurals, genitive months, 320 px layout). No review by a native speaker outside the team is recorded (issue #13)	Plural rules (one/ few/other), diacritics, case inflection
de	German	100 % (903 of 903 keys)	Martin Mudrinić (PR #8)	Checked by the team in the running app (long compounds, 320 px layout). No review by a native speaker outside the team is recorded (issue #13)	Text expansion (long compounds)

6.2 How i18n is implemented

Library / framework used	Custom typed module src/i18n (no library). Deviation from brief §2.1: the module was built in PR #8; migrating to a library (e.g. i18next or FormatJS) two days before the deadline was judged too risky for 903 keys and 166 tests. It uses the platform Intl API for all formatting and plural selection (issue #14).
Where the translation files live	src/i18n/locales/en/*.ts, hr/*.ts, de/*.ts — one folder per locale, split by area (common, compare, goals, tracking)
File format and key naming convention	TypeScript objects; semantic hierarchical keys such as tracking.activityCount, goals.monthly.description; plural messages are objects {one, few, other}; placeholders {name}
Number of translation keys	903 per locale (identical key sets, enforced by TypeScript: a missing or extra key fails npm run build)
Fallback chain	Current locale → English → the key itself

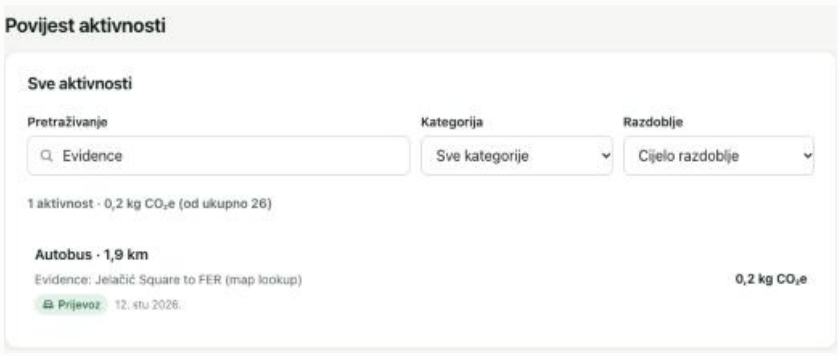
How the locale is detected and persisted	Saved choice (localStorage carbon-choices.locale; and users/{uid}.locale for signed-in users) → first supported browser language → English. Not inferred from IP.
How a missing key behaves	Development: console warning; tests fail on any missing key (54 checks render every screen in hr and de). Users never see a raw key because TypeScript requires every key in every locale.
How lang and dir attributes are set	localeStore.ts sets document.documentElement.lang on every change; dir is not set because all three locales are left-to-right.

6.3 Plural rules

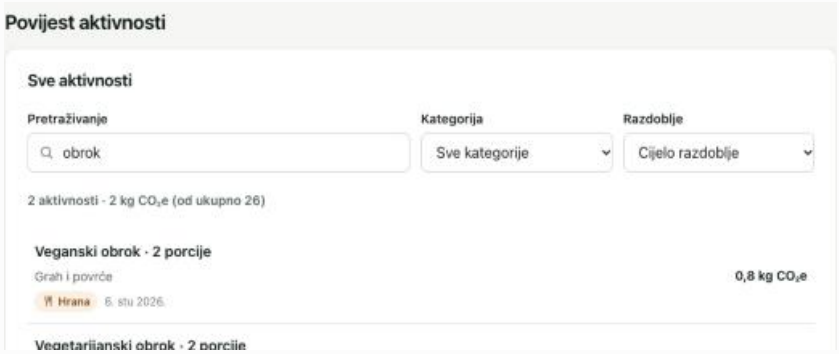
Paste the message that proves you are using real plural rules, in English and in Croatian, and show it rendering for 1, 2 and 5.

en: tracking.activityCount = { one: "{count} activity", other: "{count} activities" }

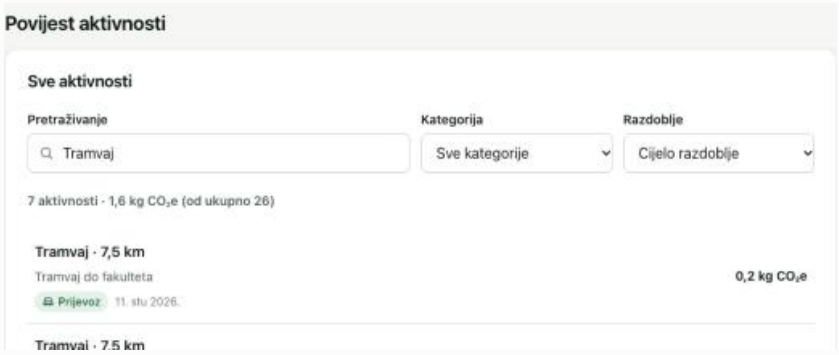
hr: tracking.activityCount = { one: "{count} aktivnost", few: "{count} aktivnosti", other: "{count} aktivnosti" }
de: tracking.activityCount = { one: "{count} Aktivität", other: "{count} Aktivitäten" }
Selected with Intl.PluralRules (src/i18n/translate.ts). Rendered by the app (formatting layer, real browser): hr 1 / 2 / 5 / 21 / 22 → 1 aktivnost / 2 aktivnosti / 5 aktivnosti / 21 aktivnost / 22 aktivnosti; en → 1 activity / 2 activities / 5 activities / 21 activities / 22 activities.



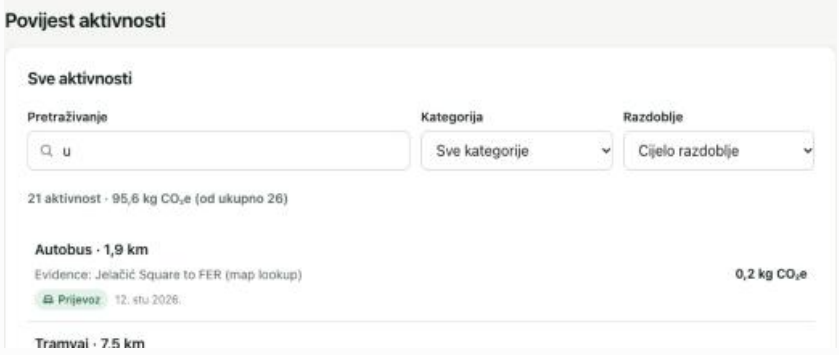
1 → "1 aktivnost" (one)



2 → "2 aktivnosti" (few)



7 → "7 aktivnosti" (other; the demo data had no filter giving exactly 5)



21 → "21 aktivnost" (one, as Croatian requires)

Figure 6.1 — plural forms: one / few / other

6.4 Locale-aware formatting

Fill in what your application actually renders in each locale — copy the values from the running product, not from the documentation.

What	en	hr	Third locale: de	Formatted by
Short date	Nov 12, 2026	12. stu 2026.	12.11.2026	Intl.DateTimeFormat (dateStyle medium)
Long date with weekday	Thursday, November 12, 2026	četvrtak, 12. studenoga 2026.	Donnerstag, 12. November 2026	Intl.DateTimeFormat (dateStyle full)
Time	Nov 12, 2026, 2:30 PM	12. stu 2026. 14:30	12.11.2026, 14:30	Intl.DateTimeFormat (timeStyle short)
Relative time ("3 days ago")	3 days ago	prije 3 dana	vor 3 Tagen	Intl.RelativeTimeFormat — not used in the interface; values shown for reference
Large number	1,234,567.9	1.234.567,9	1.234.567,9	Intl.NumberFormat
Decimal number	12.5 (kg CO_2e: 1,234.6 kg CO_2e)	12,5 (1.234,6 kg CO_2e)	12,5 (1.234,6 kg CO_2e)	Intl.NumberFormat
Percentage	47%	47 %	47 %	Intl.NumberFormat (percent)
Currency	€1,234.50	1.234,50 €	1.234,50 €	Intl.NumberFormat (currency EUR)
List of three items	bus, train, and car	bus, train i car	bus, train und car	Intl.ListFormat — not used in the interface; values shown for reference
Sorted list of names with Č, Ć, Š	Čakovec, Cres, Ćuprija, Đakovo, Dubrovnik, Šibenik, Split, Zadar	Cres, Čakovec, Ćuprija, Dubrovnik, Đakovo, Split, Šibenik, Zadar	Čakovec, Cres, Ćuprija, Đakovo, Dubrovnik, Šibenik, Split, Zadar	Intl.Collator (used on the Tracking breakdown)

6.5 Same screen, three locales

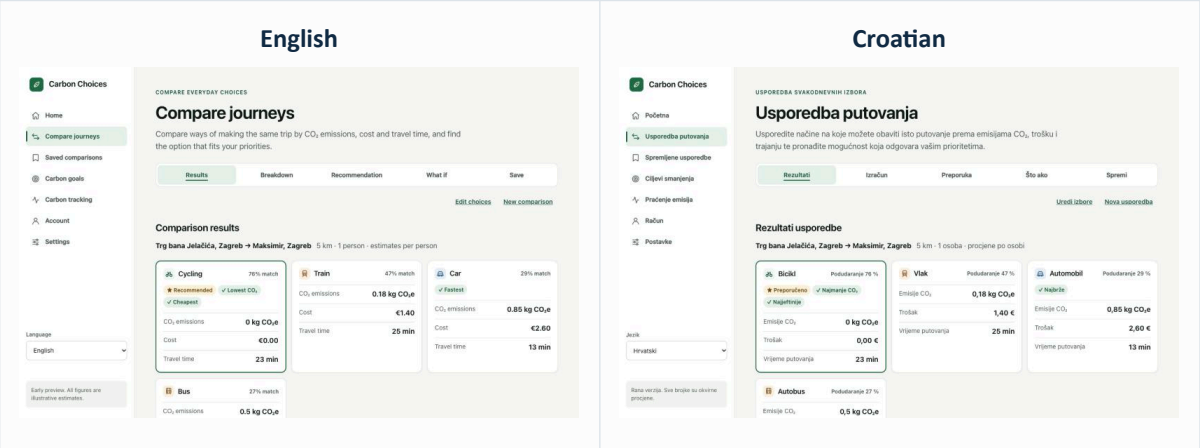


Figure 6.2 — the same screen in English and Croatian

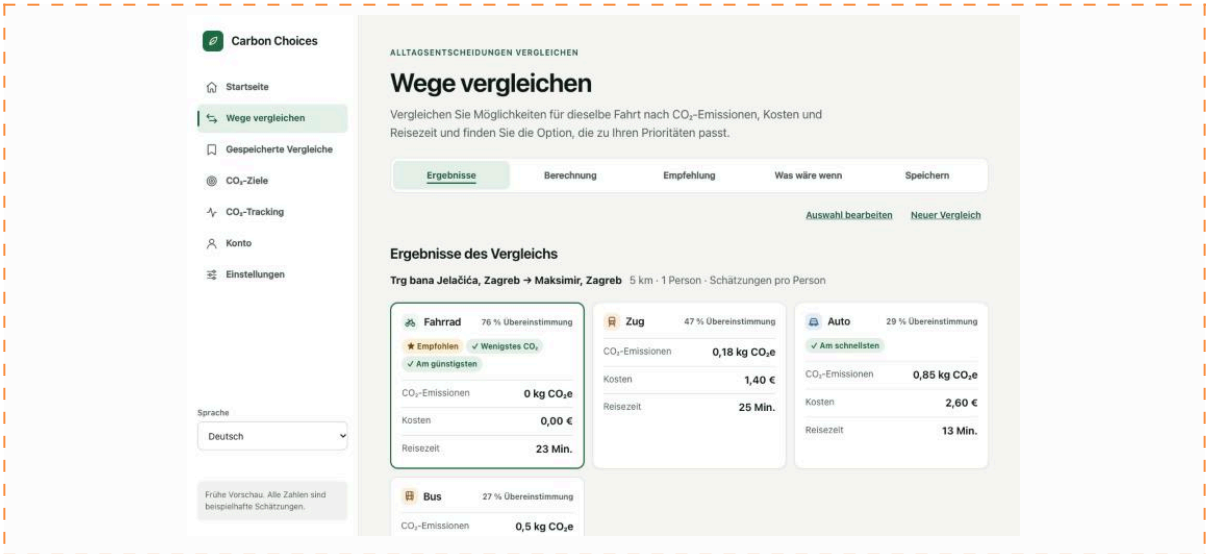


Figure 6.3 — the same screen in the third locale

6.6 Text expansion and layout

Longest locale in our interface	German (de)
Worst expansion found (label, before → after, % growth)	Tracking tab "Breakdown" → "Aufschlüsselung": 9 → 15 characters (+67 %)
What broke, and how it was fixed	At 320 px the German tab label overflowed and made the page scroll sideways (found in the requirements audit). Fixed in PR #9: tab items may shrink and long labels wrap with hyphenation ("Aufschlüsse-lung"). Checked with no horizontal scrolling at 320, 375, 640 px and desktop in all three languages.
Where truncation is still used, and why it is safe there	No truncation is used anywhere (no text-overflow/ellipsis in the CSS); long text wraps.

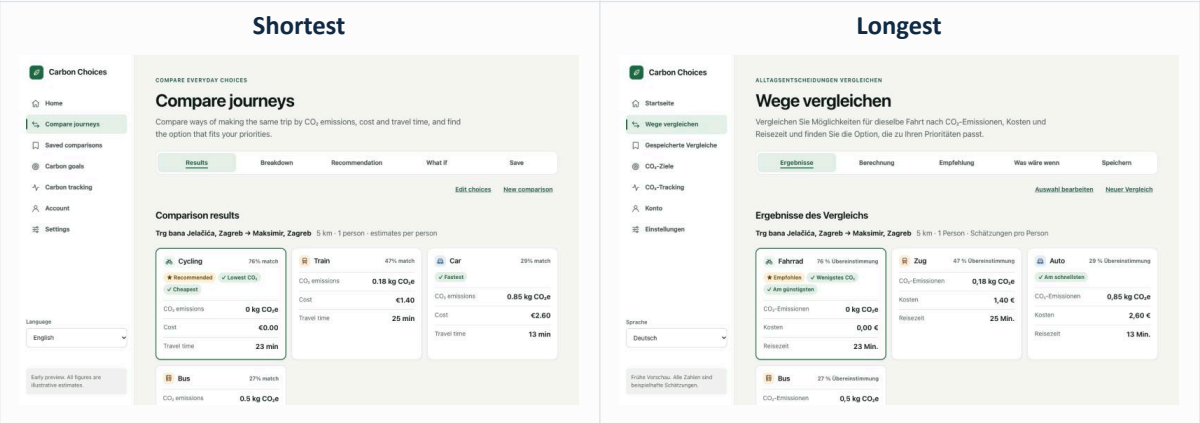
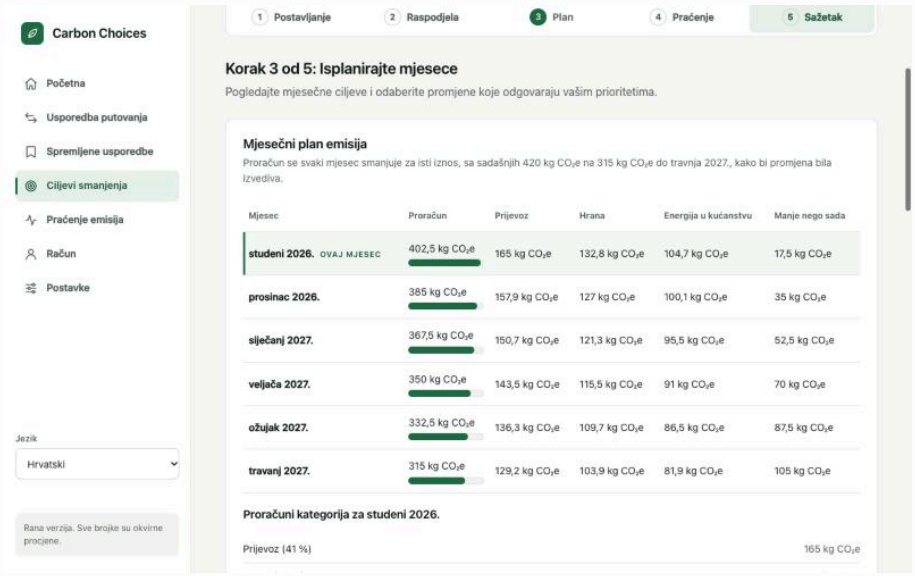


Figure 6.4 — the same layout under text expansion

6.7 Direction and script

Complete this section if your third locale is right-to-left. If it is not, state which script or line-breaking challenge you handled instead, and show the evidence.

Right-to-left locale implemented?	No
How direction is switched	Not applicable (en, hr and de are left-to-right)
Logical CSS properties used instead of left/ right?	No — the CSS uses physical left/right properties (27 places) and no logical properties; this would have to change before adding an RTL locale
Which icons are mirrored, and which are deliberately not	None mirrored; arrows point in the reading direction for LTR
How numbers and Latin names behave inside RTL text	Not applicable
If not RTL: what script or line-breaking issue was handled instead	Text expansion in German (see 6.6) and Croatian morphology: month names after prepositions need the genitive ("do travnja 2027.", "od studenoga 2026."), produced with Intl.DateTimeFormat; Croatian diacritics render in all font weights (system font).



Croatian genitive month forms in Goals ("…do travnja 2027.")

Figure 6.5 — direction and script handling

6.8 Cultural checks

Element	What we checked	What we changed
Colours that carry meaning	Status colours (green/orange/red), best-option badges, progress bars: meaning is always also given in text or a symbol (✓ Lowest CO_2, ★ Recommended, "On track")	No change needed
Icons and emoji	Every icon (transport modes, categories) is paired with a text label and hidden from screen readers	No change needed
Example data, placeholder names, avatars	Demo data uses Croatian and English notes and places (Zagreb, Rijeka, Samobor); no real people; no avatars	Added mixed-language demo data
Name, address and phone fields	The app asks only for an e-mail and free-text place names (≤ 200 characters, any script; Cyrillic and Arabic tested)	No name, address or phone fields
Anything AI-generated in the product	The product contains no AI-generated content at run time	Not applicable

7. Accessibility report

7.1 The ten-minute check

#	Check	Result	Issues found	Fixed?
1	Keyboard only — everything reachable and operable	Pass	Whole Compare journey completed with Tab, Space, arrows and Enter; delete dialog opened with Enter, closed with Esc; menu covered by tests. After a step change focus moves to the step container (no ring on that non-interactive container, by design)	n/a
2	Focus indicator always visible	Pass	2 px outline measured on every control reached in check 1	n/a
3	200% zoom / 320 px width	Pass after fix	German Tracking tab overflowed at 320 px (fixed in 18d3363); 320/375/640 px checked in en/hr/de with no horizontal scrolling. Real 200% browser zoom checked manually by the team: passed (team confirmation, 13.11.2026)	Yes (overflow, fixed)
4	Automated scan	Pass after fix	axe-core 4.14 in the browser on 7 pages + 5 Goals steps: 1 contrast violation; 0 after the fix. axe in jsdom on 27 screens: 0	Yes
5	Colour contrast	Pass after fix	Goals highlight hint 4.34:1 → 6.06:1. Placeholder and disabled states not measured separately	Yes
6	Images and alt text	Pass	No <img> elements; all SVG icons aria-hidden; every button and link has an accessible name	n/a
7	Form labels and error messages	Pass	Clicking each label focuses its field; empty submit → alert summary, field error linked with aria-describedby, aria-invalid, focus on the field	n/a
8	Heading order and lang attribute	Pass	One h1 per page, no skipped levels; <html lang> switches en/hr/de	n/a

#	Check	Result	Issues found	Fixed?
9	Screen reader — one complete task	Pass	VoiceOver on macOS: the team completed a task with the screen reader and confirmed it passed (13.11.2026). Detailed step-by-step observations were not recorded	No
10	Target size, spacing and reduced motion	Pass	Targets ≥ 44 × 44 px (CSS review); prefers-reduced-motion rules checked in styles.test.ts and with the operating-system setting by the team (team confirmation, 13.11.2026)	No

7.2 Tools used

Tool	Where it was run	Summary of result
axe-core 4.14.0 (WCAG 2.0–2.2 A/AA rules)	Real Chromium-based browser on every page and all Goals steps (2026-11-12)	1 colour-contrast violation, fixed; then 0
Vitest + jsdom + axe-core	src/test/accessibility.test.tsx: 27 rendered screens	0 violations (contrast excluded in jsdom)
Contrast calculation (WCAG formula)	src/styles/styles.test.ts on design tokens; script on the measured colours	All token text pairs ≥ 4.5:1; fixed hint 6.06:1
Manual keyboard test	Compare journey, Saved delete dialog	Pass
Manual team checks	VoiceOver (macOS), real 200% browser zoom, operating-system reduced-motion setting	Pass (team confirmation, 13.11.2026)

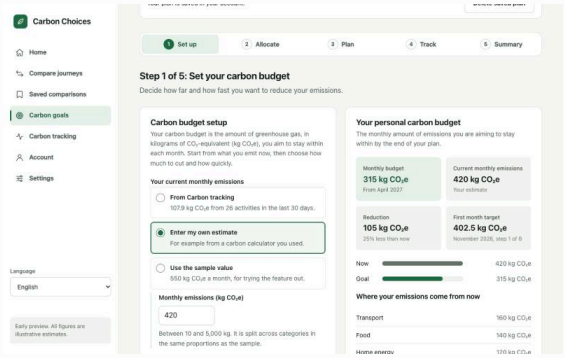
7.3 Screen reader walkthrough

Which screen reader, which task, and what actually happened. Where did you get lost, and what did you change?

Screen reader: VoiceOver on macOS. The team carried out a complete task in the running app using VoiceOver and confirmed on 13.11.2026 that it passed: the task could be completed without sighted help. Detailed step-by-step observations (exact announcements, timings) were not written down at the time, so none are reported here. What supports the result in the code: every page has one h1 and landmark regions with a skip link; all controls have accessible names; form errors are linked with aria-describedby and announced through role="alert"; loading and saved states use role="status"; the language of the page (<html lang>) follows the selected locale so VoiceOver reads Croatian and German with the right voice. Not covered: NVDA, JAWS and mobile screen readers (iOS VoiceOver, Android TalkBack) were not tested.

7.4 Contrast evidence

Measured



Measured in the browser: hint text was #66726a on #e6f1ea = 4.34:1 (fails 4.5:1)

Fixed

Fixed (commit 18d3363): the hint in the highlighted stat ("From April 2027" in the screenshot) now uses the muted text colour #4f5c54 on #e6f1ea = 6.06:1. axe-core in the browser reports 0 violations after the fix.

Figure 7.1 — contrast checks

7.5 Known accessibility issues we did not fix

An honest list scores better than an empty one. Link each to a GitHub issue.

Issue	Who it affects	Severity	Why it is still open	Issue link
Only VoiceOver on macOS tested; no NVDA, JAWS or mobile screen reader	Blind and low-vision users on other platforms	Medium	Test with NVDA and a phone screen reader in the next iteration	#10
Placeholder and disabled-state contrast not measured separately	Low-vision users	Low	Design tokens pass 4.5:1; axe-core reports 0 violations	—
Translations not reviewed by a native speaker outside the team	Croatian and German users	Medium	Team-checked; English always available	#13
No full WCAG 2.2 AA audit	All users with disabilities	Low	Ten-minute check and automated scans only; no conformance claim	—

7.6 Inclusive design decisions

Where did the persona spectrum, the social model of disability or the curb-cut effect change a decision in this product? Name the decision, not the concept.

Never colour alone (social model / curb-cut): best-option badges, Goals pace status and Tracking categories always carry text or a symbol, which also helps users on bright phone screens.

Plain-language estimates: every figure is labelled as an estimate with the formula shown (Compare breakdown, Tracking details), so users with low numeracy or low trust can check the reasoning.
Typing always works: the map lookup is optional; users who cannot or do not want to send place names to a third party, or who are offline, can type the distance.
Persona spectrum — temporary limitation: 44 px targets and a 320 px layout work for one-handed phone use on a tram as well as for users with motor impairments.
Language follows the person: the chosen language is saved in the account, so a Croatian student using a shared English computer gets their own language after signing in.

8. Git history and teamwork

8.1 Our branching model

main is protected by the GitHub ruleset "Protect main": changes only through a pull request with 1 approving review (stale approvals dismissed, the last push must be approved), no force pushes, no deletion. Each piece of work had its own branch: feature/frontend-setup, feature/carbon-reduction-goals, feature/personal-carbon-tracking, feature/compare-everyday-choices, feature/goals-integration-polish, feature/tracking-integration-polish, feature/shared-backend-database-api, feature/localization-accessibility, docs/deliverable-2-requirements-audit. There was no develop branch. The PR author merged after another member's approval. Merge conflicts were avoided by giving each member their own feature folder (src/features/<area>) and keeping shared-file changes small. Weaknesses: the reviews were approvals without comments, and PRs were merged minutes after opening (see 8.4 and 8.5).

8.2 Repository evidence

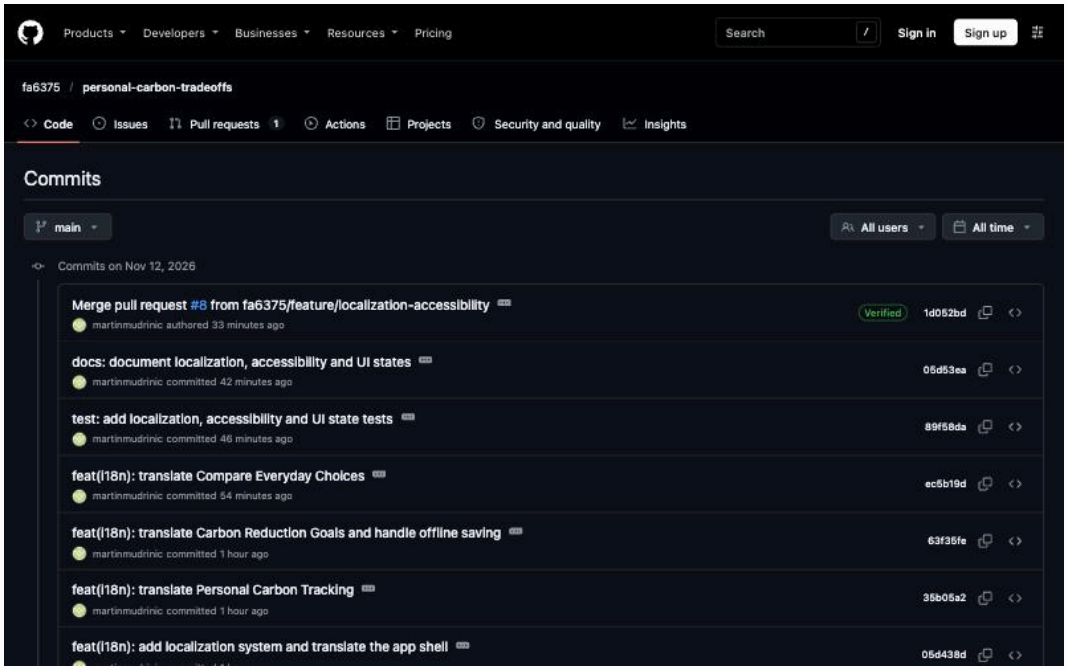


Figure 8.1 — branch and merge history

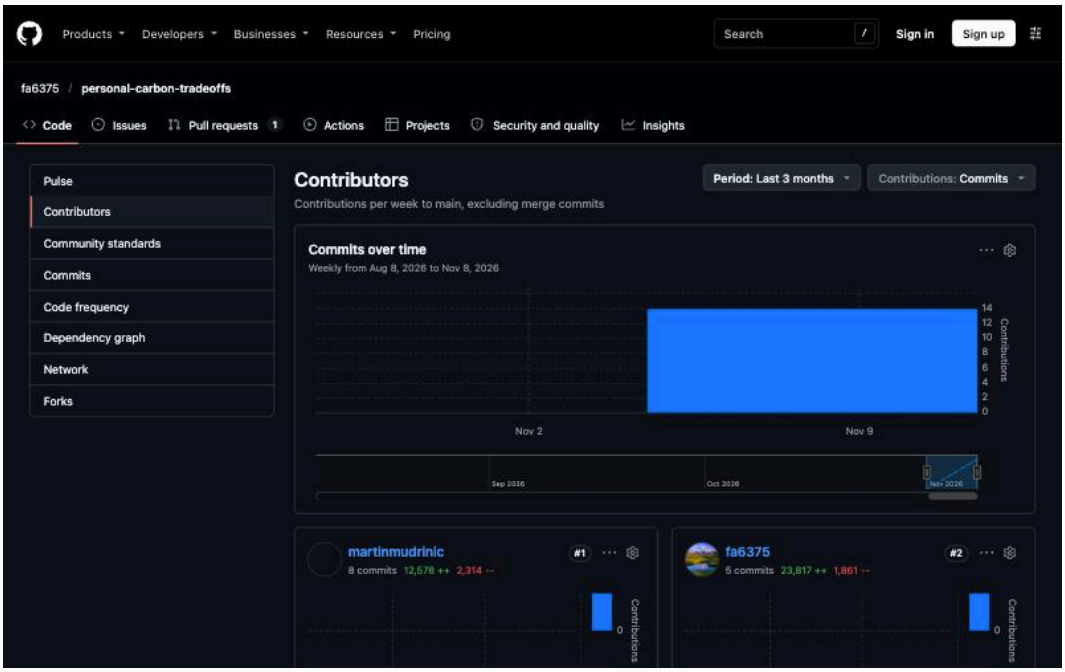


Figure 8.2 — contribution over time

8.3 Commits per member per week

Fill from the repository insights (Insights → Contributors, or git shortlog). Write each member's name in the header.

Week	Faraj Aliyev	Martin Mudrinić	Lazar Jovović	— (team of 3)	—
W1 (19.10–25.10)	0	0	0	—	—
W2 (26.10–01.11)	0	0	0	—	—
W3 (02.11–08.11)	0	0	0	—	—
W4 (09.11–14.11)	15 (9 through PR #9, 1 signing commit)	8	2	—	—
W5	—	—	—	—	—
W6	—	—	—	—	—
W7	—	—	—	—	—
W8	—	—	—	—	—
W9	—	—	—	—	—
W10	—	—	—	—	—
W11	—	—	—	—	—

Week	Faraj Aliyev	Martin Mudrinić	Lazar Jovović	— (team of 3)	—
W12	—	—	—	—	—
Total	15	8	2	—	—

8.4 Pull requests

PR #	Title	Author	Reviewer	What the review changed
#1	feat: set up frontend foundation and project README	fa6375	martinmudrinic	Approved without comments; no changes
#2	feat: implement carbon reduction goals feature	martinmudrinic	fa6375	Approved without comments; no changes
#3	feat: implement personal carbon tracking	lj7829	martinmudrinic	Approved without comments; no changes
#4	feat: implement Compare Everyday Choices	fa6375	martinmudrinic	Approved without comments; no changes
#5	fix: polish goals feature integration	martinmudrinic	fa6375	Approved without comments; no changes
#6	fix: improve tracking state and sync activities across tabs	lj7829	fa6375	Approved without comments; no changes
#7	feat: integrate shared Firebase database and external API	fa6375	martinmudrinic	Approved without comments; no changes
#8	Feature/localization accessibility	martinmudrinic	fa6375	Approved without comments; no changes
#9	Deliverable 2: audit, fixes, diagrams, evidence and Part B report	fa6375	martinmudrinic (lj7829 also requested)	Approved without comments; merged 13.11.2026

8.5 Honest account of the collaboration

What happened: the shared repository was created only on 9 November 2026, in the last week of the development period, so weeks 1–3 have no repository commits from anyone. All implementation was done between 9 and 12 November. We do not present any earlier work as commits, and no commit has been backdated.

What worked: one feature folder per member meant parallel work without merge conflicts; protecting main from the start forced every change through a pull request; integration PRs (#5, #6, #7, #8) connected the three areas into one product.

What did not work: reviews were approvals without comments, often within minutes of opening, which the brief counts as no review; the load was uneven (commit counts 15/8/2); GitHub Issues were only created at the end (#10–#17).

AI assistance: all three members used Claude Code to generate application code, which each member reviewed and tested; Claude Code also drafted and formatted the documentation from the team's instructions (see §12).

What we would set up differently: create the repository and issues in week 1, commit small pieces weekly, require a written review comment on every PR, and run the accessibility and locale checks continuously instead of at the end.

All three members confirm this account by signing the team declaration in §12.

9. Weekly progress log

One row per week per member, filled in during the week rather than reconstructed at the end. Keep it short: what was built, and the branch or pull request where it lives.

Week	Member	What was built or decided	Branch / PR	Blocked by
W1	Faraj Aliyev	No repository activity: the shared repository was created on 9.11.2026. No other verifiable record of work this week is claimed	—	—
W1	Martin Mudrinić	No repository activity: the shared repository was created on 9.11.2026. No other verifiable record of work this week is claimed	—	—
W1	Lazar Jovović	No repository activity: the shared repository was created on 9.11.2026. No other verifiable record of work this week is claimed	—	—
W2	Faraj Aliyev	No repository activity: the shared repository was created on 9.11.2026. No other verifiable record of work this week is claimed	—	—
W2	Martin Mudrinić	No repository activity: the shared repository was created on 9.11.2026. No other verifiable record of work this week is claimed	—	—
W2	Lazar Jovović	No repository activity: the shared repository was created on 9.11.2026. No other verifiable record of work this week is claimed	—	—
W3	Faraj Aliyev	No repository activity: the shared repository was created on 9.11.2026. No other verifiable record of work this week is claimed	—	—
W3	Martin Mudrinić	No repository activity: the shared repository was created on 9.11.2026. No other verifiable record of work this week is claimed	—	—
W3	Lazar Jovović	No repository activity: the shared repository was created on 9.11.2026. No other verifiable record of work this week is claimed	—	—
W4	Faraj Aliyev	Repository, frontend foundation and README (9.11); Compare Everyday Choices, 10 screens (10.11)	feature/frontend-setup #1; feature/compare-everyday-choices #4	—

Week	Member	What was built or decided	Branch / PR	Blocked by
W4	Faraj Aliyev	Firebase Auth + Firestore, rules (deployed), CRUD repositories for all features, OpenStreetMap API, browser-data import, emulator and live tests (12.11)	feature/shared-backend-database-api #7	—
W4	Faraj Aliyev	Requirements audit; contrast, German overflow and Croatian grammar fixes; locale saved in profile; non-Latin test; demo seed; ERD, architecture, evidence; this report (12.11)	docs/deliverable-2-requirements-audit #9 (merged 13.11)	—
W4	Martin Mudrinić	Carbon Reduction Goals, 10 screens (10.11); goals integration with Tracking data (11.11)	feature/carbon-reduction-goals #2; feature/goals-integration-polish #5	—
W4	Martin Mudrinić	Localization en/hr/de, accessibility, UI states and 145 tests (12.11)	feature/localization-accessibility #8	—
W4	Lazar Jovović	Personal Carbon Tracking, 10 screens (10.11); tracking state and cross-tab sync (12.11)	feature/personal-carbon-tracking #3; feature/tracking-integration-polish #6	—
W4	Lazar Jovović	Requested reviewer of the final report, README and evidence; signed the report (13.11)	docs/deliverable-2-requirements-audit #9	—

10. Testing, quality and known issues

10.1 How the product was tested

Type	What was covered	Where it lives / how to run it
Automated tests (if any)	npm test: 166 tests in 11 files (unit, localization, components, axe, UI states, CSS). npm run test:emulator: 20 tests (11 security-rules, 9 auth/ repository incl. Croatian, Cyrillic and Arabic text). npm run test:live: 7 tests against the real project (rules deployed). All passed again on 13.11.2026 (00:15 CET) on commit 799016b; build and lint pass; a fresh clone also installs, builds and passes npm test.	src/**/*.test.ts(x), tests/emulator, tests/live; logs in docs/evidence/ 19a–19f
Manual test pass	Every screen of all three features with demo data: create/edit/delete activity, save/open/delete comparison, goal plan autosave and delete, data import, distance lookup (working and failing), reload persistence, second user cannot see first user's data	Real browser; screenshots in docs/ evidence
Cross-browser / device check	Chromium-based browser at 320, 375, 640 px and 1280 px; VoiceOver check on macOS by the team. Firefox and a real phone were not tested	docs/evidence/13-…
Locale check	All features in en, hr, de: plurals (1/2/7/21), formatting, persistence after reload, profile language after sign-in, no overflow at 320 px	docs/evidence/09–12
Accessibility check	Ten-minute check, all 10 items (see §7.1); items 9 and 10 and real 200% zoom confirmed manually by the team	§7

10.2 Known bugs and limitations

Issue	Impact on the user	Workaround	Issue link
Screen reader tested with VoiceOver on macOS only	Possible problems with NVDA or phone screen readers	—	#10
Translations not reviewed by a native speaker	Possible unnatural Croatian/ German phrasing	English is always available	#13

Issue	Impact on the user	Workaround	Issue link
Custom i18n module instead of a library	None for users; maintenance risk	—	#14
Bundle about 1.1 MB (325 kB gzipped)	Slower first load on slow mobile networks	Cached after first load	#15
No password reset or e-mail verification	A user who forgets the password cannot recover the account	Create a new account	#16
Public OpenStreetMap services without uptime guarantee; ambiguous names find the first match	Lookup may fail or pick the wrong place	Type the distance; check the place names shown	—

10.3 What we would do next

1. **Native-speaker review and wider screen-reader testing** (#13, #10) — an outside Croatian and German reviewer, plus NVDA and phone screen readers, test what the team could only check itself.
2. **Move to an established i18n library and split the bundle by page and locale** (#14, #15) — lowers the maintenance risk of the translation layer and halves first-load time on phones.
3. **Replace illustrative factors with an emission-factor API behind a Cloud Function** (e.g. Climatiq), plus password reset (#16) — makes the numbers trustworthy once the core is verified.

11. Individual contribution statements

One block per team member, written by that member. The claims here must be traceable in the repository.

Member 1

Name	Faraj Aliyev
GitHub username	fa6375
What I built (features, files, modules)	I set up the repository and the shared frontend foundation (app shell, hash routing, layout, navigation, design tokens and reusable components) and built Compare Everyday Choices: all ten screens, the emissions, cost and travel-time calculations, the priority-weighted recommendation and saved comparisons (src/features/compare). I built the shared backend: Firebase Authentication with email and password, Cloud Firestore repositories with create, read, update and delete for activities, comparisons and goal plans, the owner-only security rules and their deployment (firestore.rules, src/features/*/repository.ts, src/lib/firebase.ts), and the import of data saved in the browser. I integrated the external OpenStreetMap API (Nominatim geocoding and OSRM routing) with throttling, caching and typed error handling (src/features/routing). I wrote the emulator and live test suites, debugged the integration of the three features, and in PR #9 fixed the contrast, German overflow and Croatian grammar issues, stored the locale in the user profile, added the non-Latin test, the emulator demo seed, the ERD and architecture diagram, and assembled this report.
My commits / PRs that show it	PRs #1, #4, #7 and #9 (all merged). Commits c43097e, 4508455, 4d3e610, a103487, 877dfcb, 18d3363, 5cffb49, f04557a, 1d9b96b, 7e47321, 704bc5a, 799016b and the final report commit in PR #9. Much of this code was produced with Claude Code and checked by me (see §12).
What I learned that I did not know before	Writing Firestore security rules that validate field names, types and ranges, and testing them against the emulator before deploying; handling the failure modes of a free public API (rate limits, timeouts, empty and malformed answers) so the user always has a way forward; and keeping a real project and a demo project strictly apart.
Weeks I could not contribute, and why	The shared repository was created on 9 November 2026, so weeks 1–3 of the development period (19.10–08.11) show no repository work from me or anyone else in the team. I do not claim any work in those weeks that the record cannot show. All of my repository work is from week 4.
Signature / date	Signature: AFaraj Date: 13 November 2026

Member 2

Name	Martin Mudrinić
GitHub username	martinmudrinic
What I built (features, files, modules)	I built Carbon Reduction Goals & Recommendations: the ten screens grouped into five steps, the monthly carbon budget and allocation arithmetic, the reduction ideas and plan adjustments, and the integration with Carbon tracking data for the baseline and progress (src/features/goals). I built the localization of the whole application in English, Croatian and German: the typed message catalogues, plural selection with Intl.PluralRules, Intl number and date formatting, the language selector and the <html lang> switch (src/i18n). I improved accessibility across the app and added the shared loading, empty, error and offline states, with localization, accessibility and UI-state tests (src/test).
My commits / PRs that show it	PRs #2, #5 and #8. PR #2 contains the Carbon Reduction Goals feature, PR #5 includes improvements to goal calculations and integration with tracking data, and PR #8 covers localization, accessibility and shared application states. These changes are available in the GitHub repository and show my contribution to the project.
What I learned that I did not know before	I learned how to organize a carbon reduction goals feature, calculate monthly budgets and connect goals with tracked activities. I also gained experience with localization, plural rules, date and number formatting, accessibility and handling loading and error states in a React application.
Weeks I could not contribute, and why	The shared repository was created on 9 November 2026, so weeks 1–3 of the development period (19.10–08.11) show no repository work from me or anyone else in the team. I do not claim any work in those weeks that the record cannot show. All of my repository work is from week 4.
Signature / date	Signature: MMartin Date: 13 November 2026

Member 3

Name	Lazar Jovović
GitHub username	lj7829
What I built (features, files, modules)	I worked on the Personal Carbon Tracking feature, which includes ten screens: dashboard, adding transport, food and home-energy activities, activity history, activity details, editing and deleting activities, emissions breakdown and empty states. I worked on activity validation and CRUD functionality, allowing users to create, view, update and delete their activities. I also added synchronization across browser tabs and checked that the project builds successfully and passes lint checks. My work is mainly in src/features/tracking and src/pages/TrackingPage.tsx.
My commits / PRs that show it	PRs #3 and #6. Commits 54b98fa and 50c20e2. The Tracking implementation in PR #3 was prepared with AI assistance (Claude Code) on Faraj's computer and handed to me; I committed it and made the follow-up changes in PR #6 (see §12). The README, evidence and report files were committed from Faraj's account in PR #9, where I am a requested reviewer.
What I learned that I did not know before	Modelling activities in three categories with category-specific fields and illustrative emission factors, keeping state consistent across browser tabs, and documenting a project so that someone outside the team can run it.
Weeks I could not contribute, and why	The shared repository was created on 9 November 2026, so weeks 1–3 of the development period (19.10–08.11) show no repository work from me or anyone else in the team. I do not claim any work in those weeks that the record cannot show. All of my repository work is from week 4.
Signature / date	Signature: Jlazar Date: 13 November 2026

Member 4

Name	Not applicable — the team has three members
GitHub username	—
What I built (features, files, modules)	—
My commits / PRs that show it	—
What I learned that I did not know before	—
Weeks I could not contribute, and why	—
Signature / date	—

Member 5

Name	Not applicable — the team has three members
GitHub username	—
What I built (features, files, modules)	—
My commits / PRs that show it	—
What I learned that I did not know before	—
Weeks I could not contribute, and why	—
Signature / date	—

12. AI usage declaration

Declare every AI assistant used for code, translation, documentation or design during this deliverable. Declaring use costs nothing; undeclared use that surfaces during the presentation costs a great deal.

Tool / model	Used for what	Who used it	How the output was verified
Claude Code (Anthropic; Claude Opus model)	Coding: generating application code — Compare Everyday Choices, the frontend foundation and the shared backend (Faraj; PRs #1, #4, #7, #9), Carbon Reduction Goals, localization and UI states (Martin; PRs #2, #5, #8), Personal Carbon Tracking (Lazar; PRs #3, #6; the PR #3 implementation was generated on Faraj's computer and handed to Lazar), and the tests	Faraj Aliyev, Martin Mudrinić, Lazar Jovović	Each member reviewed the code for their own feature; build, lint and all test suites pass (166 unit/component, 20 emulator, 7 live); manual checks in the browser
Claude Code (Anthropic)	Debugging and testing: Firebase package conflicts, emulator configuration, rules tests, the non-Latin and profile-locale tests, the browser axe-core scans and keyboard checks	Faraj Aliyev	Tests run and their logs saved in docs/evidence/ 19a–19f; failures reproduced before and after each fix
Claude Code (Anthropic)	Localization fixes: Croatian genitive month names, rewritten Croatian goal sentences and Intl.Collator sorting (PR #9)	Faraj Aliyev	Checked in the running app in Croatian (screenshot 06e) and by unit tests; no outside native-speaker review (#13)
Claude Code (Anthropic)	Documentation: drafted, structured and formatted this report, the README, the requirements audit, the ERD and architecture diagrams and the evidence pages, from the team's instructions and the project facts	Faraj Aliyev, for the team	The team decided the content and checked every claim against the code, test logs, screenshots and GitHub records; each member reviewed and signed their own statement

We confirm that every member of the team can explain any part of the submitted code, that the git history has not been fabricated or backdated, that no secrets are committed to the repository, and that third-party code and libraries are attributed in the README.

Name	Signature	Date
Faraj Aliyev	AFaraj	13 November 2026
Martin Mudrinić	MMartin	13 November 2026
Lazar Jovović	Jlazar	13 November 2026

13. Submission checklist

☐ Repository link works, instructor has access, release tagged — *the repository is public and the link works; release tagged v1.0-deliverable2; the invitation to ar-rit is still waiting for the instructor to accept it*

☒ README lets a stranger run the project on a clean machine — *verified on a fresh clone: npm install, build, lint and 166 tests pass; npm run demo needs no Firebase account*

☒ No secrets in the repository; .env.example present

☐ Every member has commits in most weeks of the development period — *no: weeks 1–3 have no commits (see §8.5)*

☐ Branches used, pull requests reviewed by someone other than the author — *feature branches and approval by another member on every PR, but the reviews had no written comments (see §8.5)*

☒ Database schema and migrations in the repository, seed data loads — *schema = firestore.rules + TypeScript types (Firestore needs no migrations); seed data loads with npm run demo*

☒ ERD matches the implemented schema

☒ External API integrated, failure modes handled, attribution where required

☒ All user-visible strings externalised; no hardcoded text anywhere

☒ Three locales complete, including plurals and locale-aware formatting — *team-checked; no outside native-speaker review (#13)*

☒ Text expansion and direction (or script) handled, with screenshots — *expansion handled at 320 px; all three locales are left-to-right*

☒ Ten-minute accessibility check completed and reported, remaining issues logged

☒ Screen map traces every screen back to a Deliverable 1 wireframe

☒ Weekly log filled in, individual contribution statements signed — *statements signed by all three members on 13.11.2026*

☒ AI usage declared

☒ Exported to PDF, named Team02_UX_Deliverable2.pdf — *in the repository at docs/Team02_UX_Deliverable2.pdf*